

Article

A Novel Multimodal Hierarchical Large Language Model for Enhancing Image and Text Sequence Recommendations in Item and User Modeling

Wen-Long Dong , Richard Tai-Chiu Hsung ^{*} , Harris Sik-Ho Tsang , Tony Yulin Zhu  and Wai-Lun Lo 

Department of Computer Science, Hong Kong Chu Hai College, Hong Kong, China;
246360206@student.chuhai.edu.hk (W.-L.D.); harristsang@chuhai.edu.hk (H.S.-H.T.);
ylzhu@chuhai.edu.hk (T.Y.Z.); wlo@chuhai.edu.hk (W.-L.L.)

* Correspondence: richardhsung@chuhai.edu.hk

Abstract

Existing e-commerce recommender systems, like graph-based retrieval and user behavior-driven Collaborative Filtering, usually take either textual or visual features for item recommendations to users. Traditional hierarchical large language models (HLLM) also suffer from high computational overhead for long user sequences and insufficient cross-modal semantic alignment. In this paper, we propose a multimodal hierarchical large language model (MHLLM) recommender system that leverages user behavior and integrates large language models to enhance text–image search precision and commercial value. Our MHLLM is a two-stage V-shaped model. The model decouples multimodal feature modeling and user behavior modeling: the first stage leverages Item-LLM and Item-CLIP to extract text semantics and cross-modal visual-text features respectively; the second stage adopts a learnable dynamic gating mechanism to adaptively fuse dual-modal features and uses User-LLM to model user preferences for personalized recommendation. We further design a multi-phase training strategy and lightweight feature projection structure to solve the gradient vanishing problem in joint training and reduce computational cost. Experimental results show that our MHLLM significantly outperforms traditional systems, improving key metrics like Recall@5 and NDCG@5 over 20% in most cases. Additionally, by efficient leveraging LLM and CLIP, MHLLM reduces the repeated inference overhead of User-LLM by over 30% via feature caching. Ablation experiments verify the effectiveness of core components such as dynamic gating, gating smoothing loss and task-specific prompts. The proposed model balances recommendation accuracy and deployment efficiency and has practical application value for large-scale e-commerce image-text retrieval and recommendation scenarios.



Academic Editors: Xianyu Zhang and
Domenico Ursino

Received: 25 January 2026

Revised: 9 July 2026

Accepted: 23 July 2026

Published: 29 July 2026

Copyright: © 2026 by the authors.
Licensee MDPI, Basel, Switzerland.
This article is an open access article
distributed under the terms and
conditions of the [Creative Commons
Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: large language model; contrastive learning; machine learning; recommender systems; cross modal retrieval

1. Introduction

Global retail e-commerce sales are projected to reach 5.8 trillion in 2023 and exceed 8 trillion by 2027 (Statista [1]). While growth expands consumer choice, it also creates “choice overload” and the “long-tail problem,” especially in visual categories like fashion and home décor [2,3]. As a result, traditional text-based search and recommendation methods often fall short.

Consumer demand for personalized, convenient shopping is rising. IBM reports that 73% of users will pay more for a personalized recommender system (RS) [4], and over 80% want to discover products via image uploads. However, many platforms still rely on single-modal recommendations that cannot capture complex needs [5]. In a competitive landscape, personalization is a key differentiator—McKinsey notes it can cut acquisition costs by up to 50%, raise revenues by 5–15%, and improve marketing efficiency by 10–30% [6]. With 5G and AI accelerating adoption, visual search is increasingly central to e-commerce, making image–text integration essential for future recommendation systems. Conventionally, there are two main RS approaches: Content-Based Filtering (CBF) [7] and Collaborative Filtering (CF). Both of these rely on single-modal data and are limited by feature dependence, data sparsity and cold-start problems [8,9].

In recent years, deep learning-based convolutional neural networks (CNNs) [10,11], sequential models (SASRec, Gru4Rec) [12], large language model-based recommenders (HLLM) [13], and pure visual models (PixelRec) [14] have been widely applied. However, existing approaches still have three critical drawbacks:

1. Single-modal limitation: LLM-based methods only utilize text information and ignore visual features; visual-only models fail to model user historical behavior and lack personalization [14].
2. Insufficient deep fusion: Existing multimodal recommenders only adopt shallow feature concatenation without realizing in-depth semantic alignment between images and texts; generic cross-modal models such as CLIP and MagicLens cannot model user preferences [15].
3. Low computational efficiency: Most LLM-based recommenders [16,17] directly feed long user behavior sequences into models, resulting in excessive training and inference overhead.

In this paper, we propose the Multimodal Hierarchical Large Language Model (MHLLM), a novel recommender system that generates personalized product recommendations from both textual and visual user inputs. MHLLM integrates multimodal capabilities by leveraging CLIP [15] to extract rich representations from images and text and by incorporating users' interaction history to align recommendations with their preferences [17]. The contributions of this work are as follows:

1. **A Novel Architectural Design:** MHLLM employs a novel two-stage V-shaped architecture, optimized for efficient implementation and deployment of a multimodal RS. This architecture facilitates the seamless integration of Item-CLIP, Item-LLM, and User-LLM and consequently allows for efficient training and inference.
2. **Advanced Model Alignment and Optimization:** Multiple deep neural networks—Item-CLIP, Item-LLM, and User-LLM—are aligned using a *dynamic gating mechanism* that effectively integrates information from different modalities and user history. The system is trained with an additional gating loss incorporated into the optimization objective to encourage proper gate behavior.
3. **Robust Multimodal Training Data Construction:** A robust and comprehensive training dataset is constructed specifically tailored for the development and evaluation of multimodal RS. This dataset is crucial for achieving the high accuracy demonstrated by MHLLM.
4. **Improved Performance Compared to State-of-the-Art:** Extensive experiments on the Taobao Tianchi dataset [18] show that MHLLM outperforms existing state-of-the-art methods, including Byte HLLM [13] and traditional SASRec algorithms [12]. This superior performance highlights the effectiveness of our proposed architecture and training methodology.

In the proposed MHLLM, a frozen multimodal model extracts image and command features from both Item-CLIP and Item-LLM to compute item matching (search) probabilities. The User-LLM also generates recommendation probabilities for each product. Fusing these probabilities yields a final recommendation score that combines user behavior with image-search intent, producing more tailored recommendations.

In the subsequent sections, we will begin by presenting a literature review of RS in Section 2. Next, we will elaborate on our proposed MHLLM-based RS. Section 4 will provide a comprehensive performance analysis, followed by conclusions in Section 5.

2. Related Works

In the field of e-commerce, recommender systems are designed to address the challenge of matching users with relevant products. By analyzing user preferences and product characteristics, these systems aim to enhance user experience and increase the profitability of e-commerce platforms. The basic concept involves two key components: **users**, who seek personalized suggestions based on their interests, and **items**, which represent the products available for purchase. Ultimately, the goal of RS is to provide tailored recommendations that drive customer engagement and improve sales outcomes.

2.1. Conventional, Deep Learning-Based and LLM-Based Recommendation Systems

Collaborative Filtering (CF) and Content-Based Filtering (CBF) are fundamental frameworks in recommendation systems [8,9]. CF relies on user-item similarity but struggles with data sparsity and cold starts, while CBF offers personalized recommendations but is prone to overfitting and heavy feature dependence. Both methods focus solely on unimodal data, neglecting essential visual information in e-commerce.

Advancements in deep learning have transformed recommendation systems, overcoming traditional modeling limitations [19]. Current deep learning-based models fall into three categories: DNN-based models [20–24], GNN-based models [25–27], and multimodal session-based recommendation models (MMSBR) [5]. DNN models, like Wide & Deep [21] and Deep Interest Network (DIN) [22], and deep interest evolution network (DIEN) [23] combine shallow linear modules and deep non-linear modules to balance memorization and generalization by focusing on user interactions and text features. GNN models (e.g., PinSage [26], MMGCN [28]) capture item correlations, while MMSBR partially addresses unimodal gaps by fusing multimodal features for short-term intent. Yet these approaches often still underuse visual information and struggle to capture long-term user preferences, reducing consistency in multimodal e-commerce. Recent works attempt to model multi-behavior interactions via hypergraph contrastive learning [29], yet they remain largely text-centric.

In recent years, large language models (LLMs) like LLM4Rec [30] and HLLM [13] have improved recommendation systems by enhancing the extraction of product textual features and user behavior modeling. However, they suffer from a critical unimodal dependency, relying exclusively on textual information and neglecting the vital visual cues that are essential in e-commerce.

2.2. Cross-Modal Datasets and Approaches

Multimodal recommendation datasets combine images with corresponding text to support cross-modal modeling and evaluation. For example, PixelRec [14] is a large, high-quality multimodal dataset for short-video scenarios, providing original images with descriptions to train and evaluate visual RS. MegaPairs [31] generates large-scale synthetic image-caption pairs, enabling research into visual-text relationships and also serving as a valuable resource for recommendation data augmentation. Finally, MagicLens [32] uses

CLIP [15] for interactive visual exploration via natural-language queries; although designed for multimodal retrieval, its vision–language bridging can help RS tasks such as generating high-quality datasets.

In this research, a subset of the RecapDataComp-1B dataset [33] is used as the image library; three similarity models are used to mine image pairs, including a visual semantic relevance model (e.g., the image encoder of EVA-CLIP [34]), a visual pattern relevance model (e.g., the visual encoder of DINOv2 [35]), and a description relevance model (e.g., the text encoder of EVA-CLIP [34]). For each query image, these three models are used to retrieve a set of relevant images, covering a variety of relationships from semantics to vision. To further sharpen the retrieval model’s discriminative ability, every positive pair is augmented with difficult negative samples [36].

InternVL2-26B [37] is used as a multimodal large language model (MLLM) to generate a detailed description of the similarities and differences between each pair of images. LLaMA3-8B [38] is then used as a LLM to further refine the description generated by the MLLM into specific text instructions. To ensure instruction diversity, at least three different text instructions are created for each pair of images, and five difficult negative samples are introduced. We constructed a dataset comprising over 26 million triples of (query image, query sentence, and target image). The features are automatically generated and annotated using fully open-source datasets and models. This developed dataset provides a rich array of samples, both in terms of diversity and quality, ensuring effective training and demonstrating exceptional scalability.

Besides introducing a synthesized dataset, a series of neural network models have been fine-tuned and integrated in [31]. The model for Background Generation for Vision and Language (BGE-VL) is dubbed as multimodal retrieval (MMRet). It is trained on the MegaPairs dataset. This includes BGE-VL-Base (MMRet-Base, 149 M) and BGE-VL-Large (MMRet-Large, 428 M), both based on the CLIP architecture [15], as well as BGE-VL-MLLM (MMRet-MLLM, 7.57 B), which is built on a MLLM architecture [31,38]. These models achieve state-of-the-art zero-shot performance on multiple popular Combined Image Retrieval (CIR) benchmarks and Massive Multimodal Embedding Benchmark (MMEB) [39]. There are two models trained: CLIP-based MMRet and MLLM-based MMRet. The former one follows a classic dual-encoder paradigm. An image encoder Φ_I projects each input image I into a feature vector e_i , while a separate text encoder Φ_T converts the corresponding text T into an embedding e_t . These two modality-specific vectors are then fused via element-wise addition, yielding the joint representation of e_{it} , i.e.,:

$$e_{it} = \Phi_I(I) + \Phi_T(T) = e_i + e_t \quad (1)$$

Regarding MLLM-based MMRet, a single unified model is exploited to handle both visual and textual inputs through task-specific prompt. Each instance is formatted as a sequence of “<instruct> {task_inst} <query> {q_t}{q_i} [EOS]”, where {task_inst} describes the retrieval task in natural language, with {q_t} and {q_i} denoting the text and image query tokens, respectively, and [EOS] is the end-of-sequence token [31]. To train MLLM-based MMRet, the standard InfoNCE loss function [40] is adopted:

$$\mathcal{L} = -\frac{1}{|Q|} \sum_{q_i \in Q} \log \frac{\exp(e_{q_i} \cdot e_{c_i^+} / \tau)}{\sum_{c_j \in C} \exp(e_{q_i} \cdot e_{c_j} / \tau)} \quad (2)$$

where Q represents all query samples in the batch, $|Q|$ is the total count of the batch; e_{q_i} and $e_{c_i^+}$ are the embeddings of the query sample and its positive sample, respectively; C is the set of all candidate samples in the batch; and τ is the temperature hyperparameter.

2.3. Limitations of Existing Research and the Proposed Work

Current multimodal recommendation systems face three key limitations:

1. Most models adopt shallow feature fusion without deep semantic alignment between visual and textual data.
2. LLM-based systems lack critical visual information, while generic cross-modal methods ignore personalized user behavior.
3. Existing models suffer from high computational overhead and latency.

Different from simply stacking existing modules, MHLLM integrates hierarchical LLM and CLIP based on a specially designed two-stage V-shaped architecture. It realizes deep cross-modal alignment, user preference modeling, and efficiency optimization simultaneously. The Item-LLM and Item-CLIP are responsible for extracting text and visual features, respectively, and the dynamic gating mechanism completes adaptive feature fusion. Combined with the multi-phase training strategy, our model overcomes the above three limitations of existing methods.

3. Proposed Multimodal Hierarchical Large Language Model (MHLLM)

3.1. Architectural Rationale

The key methodological challenge in adopting Vision-Language Models (VLMs) for multimodal recommendation is how to integrate VLM tokens into an LLM-based framework. Naively feeding raw VLM outputs into an LLM introduces substantial computational noise and semantic misalignment, since LLMs are optimized for textual tokens rather than dense spatial visual representations. Our design directly addresses this bottleneck by introducing a hierarchical token-injection mechanism that aggregates low-level multimodal features into high-level user behavioral representations before prompting the LLM. This hierarchy is structurally necessary to preserve temporal user dynamics while staying within the LLM's context-window limits.

3.1.1. Core Design Objectives

The core design objectives of the proposed MHLLM are as follows:

1. **Overcome Unimodal Limitations:** Enable synchronous extraction and deep fusion of product textual and visual features. Leverage dual-modal information to enhance user purchase decision-making.
2. **Decouple Feature Modeling:** Separate item feature modeling from user behavior modeling to reduce computational redundancy. Avoid excessive overhead from flattened long user sequences.
3. **Achieve Adaptive Cross-Modal Fusion:** Implement efficient end-to-end inference. Balance recommendation accuracy, personalization, and computational efficiency for industrial use.
4. **Preserve Hierarchical Reasoning:** Maintain the advantages of HLLM while enhancing visual understanding. Ensure compatibility with existing text-centric recommendation systems and adapt to mixed image-text queries.
5. **Deliver Context-Aware Recommendations:** Provide personalized product recommendations to improve e-commerce conversion efficiency. Enhance user satisfaction and bridge the gap between general cross-modal learning and practical personalized systems.

3.1.2. MHLLM Overall Framework (Two-Stage V-Structure Architecture)

As previously mentioned, HLLM [13] excels at processing product text content compared to traditional recommendation system (RS) models, but it lacks the ability to collect

and extract product image information. Consequently, it falls short when users search for products or make product queries using both texts and images. To address this, our proposed MHLLM leverages the powerful reasoning capabilities of LLMs and the advanced visual understanding ability of CLIP [15] to select products that are more suited to user queries, enhancing e-commerce conversion efficiency and user satisfaction.

The framework of our proposed MHLLM is depicted in Figure 1, which follows a two-stage V-structure architecture consisting of three core functional modules and a set of fusion and prediction components, Item-LLM (textual feature extraction), Item-CLIP (cross-modal feature extraction), and User-LLM (user behavior modeling), supplemented by a dynamic gating mechanism and a cross-modal prediction module. The overall workflow adheres to a strict “feature extraction → feature fusion → user modeling → prediction output” technical logic, ensuring clear modularity and computational efficiency.

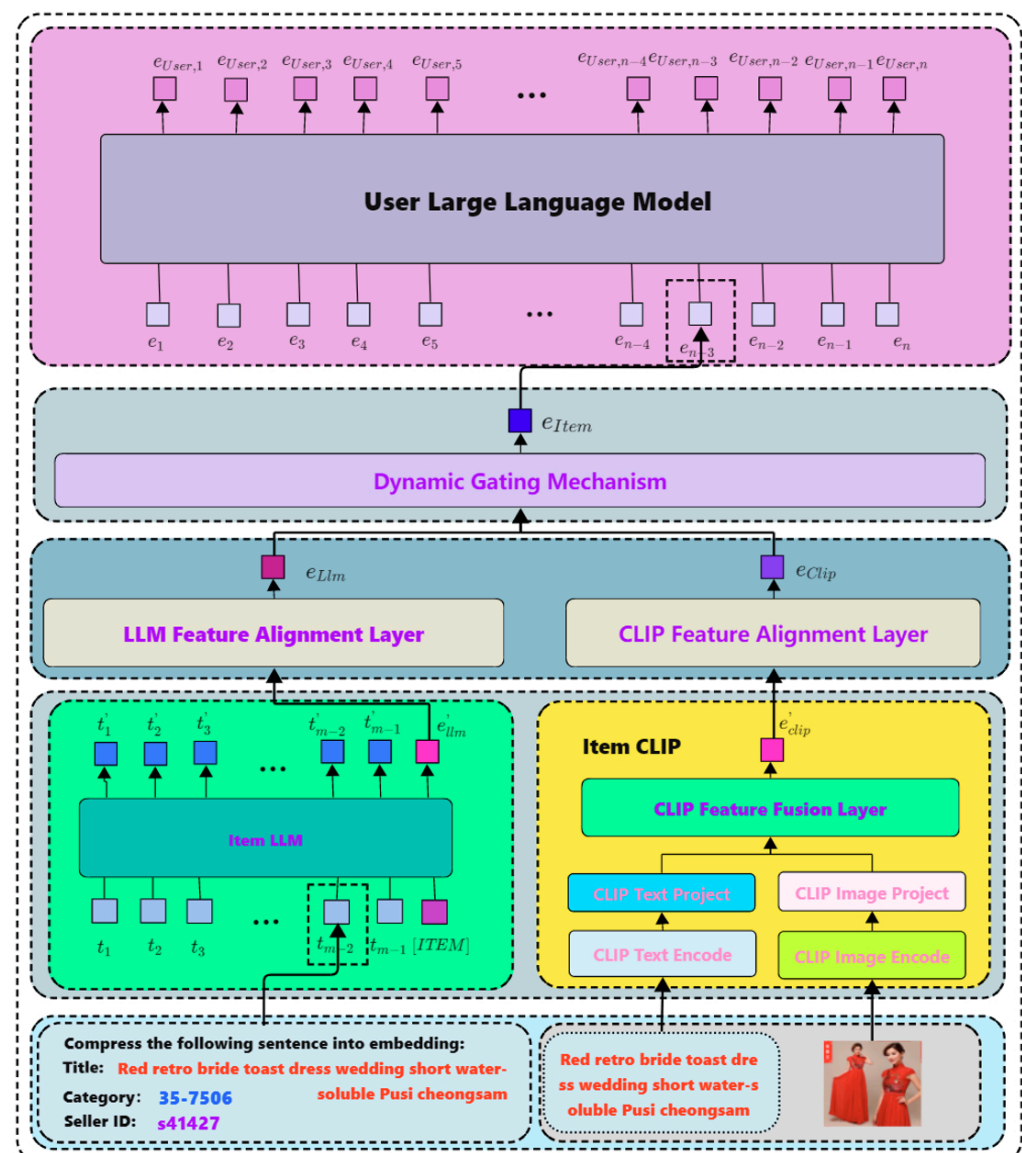


Figure 1. The proposed MHLLM architecture: product multimodal visual-text information and pure textual information are processed by Item-CLIP and Item-LLM, respectively, features are adaptively fused through the dynamic gating mechanism, and finally, user-aware recommendation prediction is completed by User-LLM based on the fused multimodal item features and user historical behavior sequences.

First, the MHLLM accepts multimodal user queries at the Item-CLIP module, generating a compact user query embedding as output. This user query embedding is then fused with the product embeddings extracted from the product database by the Item-LLM module via the proposed dynamic gating mechanism, which adaptively adjusts the contribution weight of textual and cross-modal features. The resultant fused embeddings are then fed into the User-LLM module, combined with the user's historical interaction sequence to model long-term and short-term preferences. Finally, a comprehensive embedding is obtained that encapsulates rich information from the user query, product dual-modal features, and the user's historical interactions, enabling accurate and context-aware final product prediction.

Formally, given a user $u \in U$ with a chronological historical interaction sequence $\{I_1, I_2, \dots, I_n\}$ where n denotes the length of the historical sequence, the core task of MHLLM is to predict the next interacted item I_{n+1} . Each item I_i contains its unique ID, detailed textual descriptions (e.g., product title, category, brand, specifications), and high-resolution visual information, reflecting the modern multimodal nature of e-commerce product data. This necessitates the model to jointly leverage both textual and visual information to achieve effective and comprehensive retrieval and recommendation performance, rather than relying solely on text as in traditional LLM-based recommenders.

Unlike previous LLM-based recommendation systems [41,42] that flatten user historical behaviors into plain text and feed them directly into LLMs for next-item prediction, leading to extremely long input sequences and prohibitive computational costs, the proposed MHLLM decouples item modeling from user modeling to reduce the burden of complex user sequence modeling.

In MHLLM, Item-LLM and Item-CLIP convert item images and text descriptions into fixed semantic embeddings. Then, User-LLM models user profiles and preference distributions from historical interaction sequences using these pre-extracted item embeddings. The framework is optimized for efficient information exchange across Item-LLM/Item-CLIP/User-LLM and uses a streamlined multi-phase training strategy to balance efficiency and performance. During model training, the adoption of feature pre-computation and caching mechanisms leads to an approximately 32% reduction in training time and a 35% drop in inference latency, making the model suitable for large-scale industrial deployment. Below, we provide a detailed description of each core functional module, fusion mechanism, and prediction component in strict technical order.

3.2. Core Functional Module Design

3.2.1. Item-LLM Module: Product Textual Semantic Feature Extraction

The Item-LLM module is designed to extract high-quality deep semantic representations exclusively from product textual descriptions. From a representation learning perspective, autoregressive LLMs are inherently optimized for next-token prediction rather than global semantic compression. To reorient the model toward discriminative embedding extraction, a tailored prompt acting as a task-specific **prefix conditioning instruction** is introduced, along with specialized tokenization mechanisms. This guides the model to explicitly encode product attributes, category information, and latent semantic interrelations into embedding, rather than generating open-domain text.

A customized prompt template for Item-LLM is shown in Table 1, with fixed tokens in bold and sample product data in italics. This template constrains the model to focus on feature compression and encoding, ensuring the output embeddings carry dense product textual semantics.

Table 1. Tailored prompt template for Item-LLM. Tokens and example data are in bold and italics faces respectively.

Compress the following sentence into embedding:
Title: <i>Red retro bride toast dress wedding short water-soluble Pusi cheongsam</i>
Category: <i>35-7506</i>
Seller ID: <i>s41427</i>

In this study, we employ a lightweight yet high-performance LLM architecture, TinyLLaMa-110M [43], to efficiently process diverse product text fields, including titles, detailed descriptions, tags, and category labels. Leveraging the model’s self-attention mechanism, a rich textual semantic embedding e_{LLM} is generated, which encapsulates core product characteristics and implicit attribute relationships (e.g., capturing nuanced paired attributes like “waterproof sports shoes” combined with “breathable mesh design”). The detailed processing pipeline is divided into two core stages: model positioning and input processing, and feature extraction.

(a) Model positioning and input processing

Text Preprocessing: The product’s multiple textual attributes (title, details, tags, etc.) are concatenated in order and flattened into a coherent, natural-language sentence T . To steer the model’s attention and clarify the task for the LLM, a prompt prefix such as “Compress the following sentence into embedding” is prepended to the input sequence. This prompt primes the LLM to operate in a semantic encoding mode tailored for product feature extraction rather than generative output.

Special Token Tagging: Inspired by Transformer which uses token [CLS] for classification tasks, following tokenization of the preprocessed text via the LLM’s tokenizer, we append a dedicated special token [ITEM] at the end of the token sequence, forming the complete input, i.e., $\{t_0, t_1, \dots, t_{m-1}, [ITEM]\}$, where m is the length of the textual token sequence. This tag explicitly instructs the model to focus on the product feature extraction task.

(b) Feature extraction mechanism

Leveraging the advanced semantic understanding capabilities inherent in Transformer-based LLMs, the Item-LLM module generates a high-fidelity textual feature embedding, e_{llm} , in the following manner.

Contextual Encoding: The input token sequence, $h_j^{(0)}$, with $j = 0, \dots, m$, is fed into the LLM’s Transformer encoder stacks from 1 to the l th-layer, which employ multi-head self-attention to capture complex, long-range dependencies, and contextual relationships between words. For example, in the phrase “waterproof sports shoes, breathable mesh design”, the model can discern the interplay between functional descriptors (waterproof, breathable) and categorical classification (sports shoes), effectively modeling the compositional semantics.

Feature Anchoring and Representation Extraction: Adopting a strategy analogous to the [CLS] token pooling mechanism commonly used in models like BERT [44], the output embedding corresponding to the special token [ITEM] is treated as the definitive semantic feature embedding for the product:

$$h^{(l)} = \text{Layer}_{ItemLLM}^{(l)}(h^{(l-1)}), \quad l \in [1, L]$$

$$e_{llm} = h_m^{(L)} \quad (3)$$

where $h^{(l)}$ is the l th-layer hidden state vector, and e_{llm} is the deep textual semantic embedding for the product taken from the last hidden state of the last layer of Item-LLM, $h_m^{(L)}$. With our tailored prompt and training strategy, which will be mentioned in the coming

sub-section, the product information is explicitly encoded in e_{LLM} , which is later fused with the user query embedding from Item-CLIP.

3.2.2. Item-CLIP Module: Product Cross-Modal Feature Extraction and Alignment

The Item-CLIP module, built upon the enhanced CLIP model optimized by Mega-Pairs [31], serves as a unified dual-branch image–text feature extractor dedicated to capturing and aligning rich multimodal product representations for e-commerce scenarios. This module resolves the domain gap between general cross-modal models and specialized product data, enabling robust visual–text semantic alignment.

(a) Cross-Modal Interaction and Alignment

To integrate and harmonize visual and textual modalities, the module employs the InfoNCE contrastive learning loss [40] for optimization, as shown in Equation (2). This loss function maximizes the cosine similarity between corresponding image–text pairs (positive samples) while minimizing similarity with non-matching pairs (negative samples), promoting robust and generalized cross-modal semantic alignment. Implementation details are provided in Appendix A.

Since the output embeddings from Item-LLM and Item-CLIP typically differ in feature dimensionality and representation distribution, two dedicated projection layers $\text{Proj}(\cdot)$ are introduced to unify the dimensionality of the visual embedding e_{Clip}^{Image} and textual embedding e_{Clip}^{Text} before fusion. The final cross-modal fused embedding is generated via a fixed fusion function F , formulated as

$$\begin{aligned} e_{Clip}^{Image} &= \text{ItemCLIP}_{Image}(\text{Query}_{Image}) \\ e_{Clip}^{Text} &= \text{ItemCLIP}_{Text}(\text{Query}_{Text}) \\ e_{Clip} &= F\left(\text{Proj}(e_{Clip}^{Image}), \text{Proj}(e_{Clip}^{Text})\right) \end{aligned} \quad (4)$$

where e_{Clip} represents the compact cross-modal embedding of the product, integrating both visual appearance and textual semantic information. A key design advantage is that Item-CLIP and Item-LLM remain functionally decoupled; their parameters can be frozen independently during pre-training or downstream fine-tuning stages, eliminating redundant joint tuning and significantly reducing computational overhead.

(b) Offline and Online Model Functionalities

The Item-CLIP module supports two core working modes to balance efficiency and performance for large-scale e-commerce data: **Offline Feature Pre-Extraction and Caching:** All product image and textual features are pre-extracted in bulk using the pre-trained and fine-tuned CLIP encoders, then stored in an indexed feature cache with product IDs as unique keys. This cache stores both visual and textual embeddings for rapid retrieval during training and inference, avoiding repeated feature extraction. **Online Feature Fusion and Interaction:** During runtime, cached embeddings for target products are quickly accessed via product IDs, aligned via projection layers, and fused to generate cross-modal embeddings for downstream fusion and prediction.

3.2.3. Dynamic Gating Mechanism: Adaptive Image-Text Feature Fusion

To generate a unified and comprehensive multimodal item embedding e_{Item} for downstream User-LLM modeling and final prediction, a learnable dynamic gating mechanism G is proposed to adaptively adjust the relative contribution of textual features from Item-

LLM and cross-modal features from Item-CLIP, instead of using fixed-weight fusion. The core fusion formula is defined as

$$e_{Item} = G(e_{Llm}, e_{Clip}) \quad (5)$$

The dynamic weighted gating function is designed with a two-layer multi-layer perceptron (MLP) [45] and the GELU activation function [46], which delivers superior performance in multimodal fusion tasks compared to ReLU, LeakyReLU, and SiLU (verified via ablation experiments in Section 4.4.2). The detailed calculation process is

$$\begin{aligned} z_G &= W_{G2} \cdot \text{GELU} \left(W_{G1} \cdot \begin{bmatrix} e_{Llm} \\ e_{Clip} \end{bmatrix} + b_{G1} \right) + b_{G2} \\ \alpha_{GATE} &= \begin{bmatrix} \alpha_{Llm} \\ \alpha_{Clip} \end{bmatrix} = \text{Softmax}(z_G) \\ e_{Item} &= \alpha_{Llm} e_{Llm} + \alpha_{Clip} e_{Clip} \end{aligned} \quad (6)$$

where W_{G1} , b_{G1} and W_{G2} , b_{G2} denote the weight matrices and bias terms of the first and second MLP layers, respectively. Softmax activation ensures $\alpha_{Llm} + \alpha_{Clip} = 1$ with both weights being non-negative, enabling adaptive weight assignment based on input item characteristics. Training for gating parameters W_{G1} , W_{G2} , b_{G1} , b_{G2} is used to compute α_{GATE} in Equation (6). W matrices use Xavier normal init; biases are set to 0.1. Parameters are trained end-to-end with the model under a joint loss $L_{Total} = L_{Rec} + 0.8 L_{Align} + 0.2 L_{GATE}$; L_{GATE} penalizes high variance in gating to avoid extreme modality dominance. Optimization uses AdamW (lr = 1×10^{-4} , weight_decay = 0.01) with cosine-annealed LR and 10% warm-up, distributed via DeepSpeed Stage 2. Training runs 20 epochs with early stopping (patience = 5). GELU is chosen for smooth, continuous outputs, noise suppression, and better gradients; ablations in Section 4.4 show GELU outperforms ReLU, LeakyReLU, and SiLU on Recall/NDCG.

3.2.4. User-LLM Module: User Multimodal Behavior Sequence Modeling

The User-LLM module adopts the same lightweight Llama-3-74M [43] LLM architecture to jointly model fused multimodal item features and rich user behavior data, including explicit interactions (clicks, purchases, favorites) and implicit user behavior signals (browsing duration, query frequency, skip behavior; detailed definitions in Appendix C.6). Leveraging Transformer-based self-attention mechanisms, the module captures complex non-linear interactions between user long/short-term preferences and multimodal product features, generating a semantic reasoning-based recommendation score for candidate items.

(a) User Historical Interaction Processing

As defined earlier, each user has a historical interaction sequence $\{I_1, I_2, \dots, I_n\}$. Each item in the sequence is converted into a fused multimodal embedding $e_{Item,k}$ via the dynamic gating mechanism (Equation (5)), forming a dense multimodal historical feature sequence $E_{Item,n}$:

$$\begin{aligned} e_{Item,k} &= G(e_{Llm,k}, e_{Clip,k}) \\ E_{Item,n} &= \{e_{Item,1}, e_{Item,2}, \dots, e_{Item,n}\} \end{aligned} \quad (7)$$

where $e_{Item,k}$ represents a fused embedding encoding with both textual and visual attributes for the k th item. Given this multimodal input, the User-LLM analyzes historical interactions to infer the user's latent interests and predict their engagement likelihood with subsequent candidate items.

(b) **Model Architecture and Feature Processing**

Input Layer Optimization: The standard word-embedding layer of the pre-trained LLM is discarded and replaced directly with the fused historical feature sequence $E_{Item,n}$, eliminating information loss caused by converting dense multimodal embeddings back to discrete token representations.

Pre-trained Weights and Fine-Tuning: The weights of core modules, including the Transformer encoder and attention mechanism, from the pre-trained LLM are retained. Experiments demonstrate that these pre-trained weights are essential for effective training.

Sequence Modeling and Prediction: Eventually, the model produces a predictive embedding representing the user's latent interest toward the next product:

$$e_{User,r} = UserLLM(E_{Item,r}) \quad (8)$$

where $e_{User,r}$ captures nuanced linguistic, semantic, and visual preferences from the feature sequence $E_{Item,r}$ of r items, enabling refined recommendation scores for candidate products.

3.2.5. User-LLM Variants: Early Fusion and Late Fusion for Discriminative Recommendations

After contrastive pre-training of MHLLM (using positive and negative sample pairs to learn meaningful product and user representations, detailed in Section 3.3), the pre-trained model is applied to discriminative recommendations, the dominant paradigm in industrial e-commerce, which focuses on evaluating user sequence relevance to a target item. The core task is to assess the matching degree between the user sequence $\{I_1, I_2, \dots, I_n\}$ and the target item I_{tgt} . To balance recommendation accuracy and inference efficiency, two optimized User-LLM variants are designed, early fusion and late fusion, as illustrated in Figure 2.

(a) **Variant 1: Early fusion**

In the early fusion approach, as depicted in Figure 2a, the target product features e_{tgt} , which are generated by Item-LLM and Item-CLIP, are directly appended to the end of the user's historical interaction sequence. This augmented sequence is then fed into the User-LLM. The LLM processes this combined input to generate a high-order cross-modal representation that inherently captures the complex interplay between the user's evolving interests and the specific features of the target product. This deep integration allows for fine-grained semantic alignment. The generated cross-features are subsequently mapped to an interaction probability P_{early} (or relevance score) through a dedicated prediction head, typically an MLP layer as follows:

$$P_{early} = \sigma(MLP(UserLLM([e_1, e_2, \dots, e_n, e_{tgt}])) \quad (9)$$

where σ is the sigmoid activation function. By modeling user-item semantic interactions, recommendation accuracy is significantly enhanced. However, this approach entails considerable computational costs, especially when dealing with large candidate sets.

(b) **Variant 2: Late fusion**

In the late fusion approach, as depicted in Figure 2b, a special user token [USER] is appended to the end of the user's historical sequence. The User-LLM then processes this sequence to extract a comprehensive user interest representation that is independent of any specific target product. This design allows User-LLM embedding to be pre-computed and reused. The prediction of interaction probability is then

performed by combining the independently extracted user features e_{User} and target product features e_{tgt} through an MLP layer:

$$P_{late} = \sigma(MLP([UserLLM([e_1, e_2, \dots, e_n, [USER]]); e_{tgt}])) \quad (10)$$

With the user features reused across all candidate products, the reasoning efficiency is enhanced. However, the simplistic modeling of user–product interactions may compromise the recommendation accuracy.

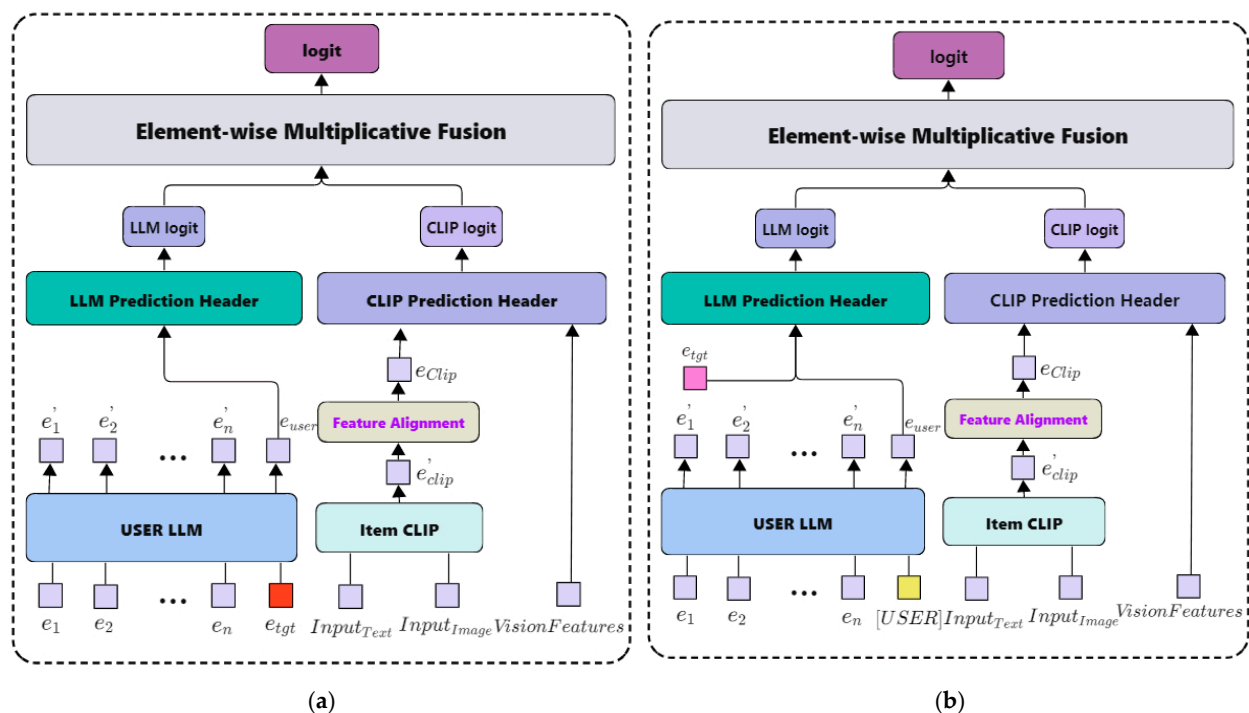


Figure 2. User-LLM variants for discriminative recommendations. (a) early fusion: the fused historical feature sequence e_n is input into the User-LLM with target product features e_{tgt} , as shown in Equation (9). The User-LLM output P_{early} is then further used to compute cross-modal joint probability with P_{Clip} ; (b) late fusion: the fused historical feature sequence e_n is input into the User-LLM with user token $[USER]$. The cross-modal joint probability is calculated from the User-LLM output with e_{tgt} .

3.2.6. Cross-Modal Prediction Module: Final Recommendation Score Generation

The cross-modal prediction module generates the final ranked recommendation list by integrating dual-modal user query information (Equation (4)) and user latent interest embeddings (Equation (8)). Specifically, the final hidden state of User-LLM is fed into a task-specific prediction head to obtain the text-centric logit P_{Llm} , while the cross-modal embedding e_{Clip} is passed into a parallel prediction head to generate the visual-centric logit P_{Clip} . The final cross-modal joint probability is calculated via the Hadamard product (element-wise multiplication) to fuse the two prediction signals:

$$P_{joint} = P_{Llm} \odot P_{Clip} \quad (11)$$

where $\odot$ represents the Hadamard product operator. For example, when a user submits a query with an image of “red sneakers” paired with the text “The hottest basketball shoes”, the visual branch captures key perceptual attributes (color, category, style), while the textual branch extracts semantic preference cues. After fusion, the combined feature vector accurately identifies products that meet both visual and semantic criteria, significantly

improving recommendation precision. Finally, candidate items are ranked by cosine similarity between their fused embeddings and the user's interest embedding, achieving joint optimization of visual semantics and user behavior modeling. The LLM and CLIP components undergo supervised fine-tuning to adapt to recommendation tasks, with detailed settings in Appendix B.

3.3. Multi-Phase Model Training Strategy

Instead of training the entire MHLLM, which demands high-performance hardware and substantial training costs and data, we implemented a carefully staged, modular training strategy. This approach enables us to develop an efficient and scalable multimodal recommendation system for large-scale e-commerce scenarios. The training pipeline emphasizes both semantic alignment and discriminative recommendation power, integrating advanced LLMs and visual encoders within a joint optimization framework. There are two major training phases, which consist of module-wise pre-training, a fine-tuning phase, and an end-to-end fine-tuning phase.

3.3.1. Module-Wise Pre-Training and Fine-Tuning Phase

First, Item-CLIP is initialized using the weights pre-trained on the MegaPairs dataset [31]. The original CLIP [15] performs well in general image–text alignment tasks which could be used as our Item-CLIP. Yet, there is a domain discrepancy when it is directly applied to e-commerce scenarios (such as product image–title matching). Item-CLIP is fine-tuned using product titles and images, which is accomplished by jointly optimized using image-text contrastive loss $\mathcal{L}_{ITC}$ and text-to-image generation loss $\mathcal{L}_{T2I}$ following [15]:

$$\mathcal{L}_{Clip} = \mathcal{L}_{ITC} + \lambda \mathcal{L}_{T2I} \quad (12)$$

where $\mathcal{L}_{ITC}$ enforce semantic similarity between images and texts, while $\mathcal{L}_{T2I}$ ensures that the text is able to accurately generate the feature representation of the corresponding image. λ is the coefficient for balancing two tasks. Positive samples are constructed using the text and image from same products while negative samples are constructed using the text and image randomly from different products. After module-wise fine-tuning, Item-CLIP is frozen as following HLLM [13].

On the other hand, Item-LLM and User-LLM are initialized using the LLM (e.g., Qwen [47] and Llama [38,48]) pre-trained on a large-scale natural language corpus. Similar to Item-CLIP, although existing LLMs have rich world knowledge and strong reasoning capabilities, they are significantly different from the cross-modal semantic alignment and user interest modeling capabilities required by RS. Thus, supervised fine-tuning (SFT) using contrastive loss $\mathcal{L}_{llm}$, which is similar to (2), is adopted onto the pre-trained LLMs. User–product interaction data (such as click, favorite, and purchase records) of e-commerce scenarios is used for SFT. The fine-tuning data uses the format of {user history sequence, current query, positive sample product, negative sample product}, which drives the model to learn the mapping relationship between user interests and product features, with the embeddings of e_{query} , e_+ and e_- derived from current query, positive sample product, and negative sample product, respectively:

$$s(e_1, e_2) = \exp(\text{sim}(e_1, e_2) / \tau) \quad (13)$$

$$\mathcal{L}_{llm} = -\log \frac{s(e_{query}, e_+)}{s(e_{query}, e_+) + \sum_{k=1}^K s(e_{query}, e_-)}$$

where τ is the temperature coefficient and K is the number of negative samples.

3.3.2. End-to-End Fine-Tuning Phase

Last but not least, a joint optimization of cross-modal comparison and recommendation objectives is performed to fine-tune Item-LLM, User-LLM and the gating network in (5). A multi-task contrastive loss is performed, which integrates NCE loss (user-item matching) $\mathcal{L}_{NCE}$, modality alignment loss (text-image consistency) $\mathcal{L}_{Align}$, and weight regularization $\mathcal{L}_{Reg}$:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{NCE} + \lambda_2 \mathcal{L}_{align} + \lambda_3 \mathcal{L}_{Reg} \quad (14)$$

where λ_1 , λ_2 and λ_3 are the weighting parameters. $\mathcal{L}_{NCE}$ is the contrastive loss, similar to (13), with the positive sample being the multimodal fusion embedding of user interaction products and negative samples being global random samples with hard negative mining (filtering samples with cosine similarity > 0.99). $\mathcal{L}_{Align}$ is the Item-CLIP modality alignment loss, which is a bidirectional cross-entropy loss to strengthen modal consistency between text and image. For discriminative recommendation, a binary classification loss function can be defined as follows:

$$\mathcal{L}_{cls} = -(y \cdot \log(x) + (1 - y) \cdot \log(1 - x)) \quad (15)$$

where y is the ground truth probability; x is the prediction probability. To further improve performance, the next product prediction loss of generative recommendation is introduced as an auxiliary item, and the final loss function is

$$\mathcal{L}_{dis} = \lambda_{gen} \mathcal{L}_{gen} + \mathcal{L}_{cls} \quad (16)$$

where λ_{gen} is the weighting parameter for optimization objective balancing. Experiments show that this strategy can improve the accuracy of product image and text alignment, and the Recall rate of cross-modal recommendation is also significantly improved.

Hence, the MHLLM facilitates a deep fusion of traditional recommendation models and visual semantics while preserving recommendation accuracy, offering a viable technical solution for personalized recommendations in large-scale e-commerce scenarios. See Appendix C for more information about the MHLLM development process. Appendix D shows characteristics, key components, and module functions of the MHLLM and other related models.

4. Experimental Results

4.1. Datasets

As the target recommendation system aims for multimodal data, an e-commerce dataset containing images is required. Given the dataset's large size, ordinary computers struggle to process it. To address this, we use Google Cloud PySpark [49] to extract a manageable subset for training.

4.1.1. Alibaba Tianchi Dataset

URL: <https://tianchi.aliyun.com/dataset/140281> (accessed on 22 July 2026)

It includes **Tianchi_2014001_Rec_Tmall_Product**, which contains 8,133,507 rows, which correspond to the attributes of 8,133,507 items on TMALL. In each row, there are six features including the item ID, title, picture, category, brand, and seller.

(a) **Tianchi_2014002_Rec_Tmall_Log**

The users' behavior of browsing TMALL reflects their preference of items. This dataset contains 1,333,729,303 rows corresponding to records of user-item interactions. Features of each row include item ID, user ID, action, and time.

(b) **Tianchi_2014003_Rec_Tmall_Review**

Review data is available for partial “user–item” pairs, which contain the review and rating on the item/merchant/logistic. This dataset contains 11,224,814 rows, corresponding to 11,224,814 reviews. Remark: A user may buy the same item multiple times; thus, there may be several reviews for a “user–item” pair. Also, a review with “n” images will be decomposed into “n” identical reviews except feature “image”. Features include item ID, user ID and feedback.

Data filtering and preprocessing:

- (a) **User Visits Filtering:** Extract records from the Tianchi_2014002_Rec_Tmall_Log table where user visits range from 5 to 10. This balance prevents reasoning difficulties and data imbalance. Retain only essential fields: item_id, user_id, and Vtime, converting Vtime to integers for easier sorting.
- (b) **Product Information:** Select product details from Tianchi_2014001_Rec_Tmall_Product, removing any without images. Save images as 12-bit data, facilitating easier retrieval.
- (c) **Data Cleaning:** Remove records with missing values from both sets to ensure data integrity.
- (d) **Dataset Generation:** From the **Alibaba Tianchi dataset**, we extracted **69,551** product entries with corresponding images, along with access logs generated by **60,052** users interacting with these products. This compiled dataset is designated as **rec_tmall_60K**.

4.1.2. Pixel Dataset

URL: <https://github.com/westlake-repl/PixelRec> (accessed on 22 July 2026)

The format of the user interaction record contains item_id, user_id and timestamp, while the product details record contains item_id, description, title and tag. The primary datasets are Pixel50K, Pixel200K, Pixel1M, and Pixel8M. Model performance is tested across these datasets, but current device limitations restrict testing to the Pixel200K dataset. Additionally, the dataset includes all 408,374 images from PixelRec. The Pixel200K dataset contains 96,121 items, 200,001 users, and 1,048,576 interactions, yielding an average of 5.24 interactions per user and a sparsity of 99.9945%.

4.2. Models Training Procedures

In the third step, we freeze the model weights and extract text and image features, saving them in text_feature_cache.bin and image_feature_cache.bin for faster access later. The fourth step trains the MHLLM using a cleaned Tmall e-commerce dataset, optimizing model weights and validating them. Finally, we conduct comparative experiments to assess performance improvements over similar models. The specific foundation models adopted in the study are Llama-3-74M [38] and BGE-VL-large [31] (pre-trained on MegaPairs).

The MHLLM training employs two acceleration modes: DeepSpeed [50] and DDP [51]. DeepSpeed enhances efficiency and scalability for large models, utilizing optimization techniques for high throughput and reduced training costs while ensuring accuracy. Its user-friendly API integrates easily with PyTorch (version 2.5.1+cu121). DDP facilitates distributed training by dividing data into batches across GPUs, synchronizing gradients for consistent model parameters.

The MHLLMs are trained using MegaPairs [31] as the underlying CLIP model. For the Pixel200K dataset, training was conducted over approximately 3 days using four NVIDIA GeForce RTX 4090 (48 GB) GPUs with the DeepSeek-R1-DRAFT-Qwen2.5-0.5B as the LLM. In contrast, training with the rec_tmall_60K dataset was performed on a single NVIDIA GeForce RTX 4090 (24 GB) GPU, taking around 3 h for the Llama-3-74M model and 2 days for the tinyllama-110M model. The complete source code for model training and evaluation is available in the Supplementary Materials.

4.3. Comparative Experimental Results

We conduct experiments to assess the performance of various recommendation models. The evaluation metrics include Recall and Normalized Discounted Cumulative Gain (NDCG), where we calculate Recall@5, 10, 50, and 200, as well as NDCG@5, 10, 50, and 200. Recall@k (similar for NDCG) specifically measures the proportion of relevant items present among the top k predictions made by each model. The model configuration parameters are shown in Table 2. Table 3 shows the corresponding access link of the models. All experiments adopt the full-item ranking protocol [14,15], where models rank the ground-truth item among the complete catalog of candidate items (69,551 for rec_tmall_60K; 96,121 for Pixel200K). This setting reflects real-world deployment scenarios.

Table 2. Model configuration parameters.

Modules	HLLM	MHLLM
Item-LLM	Llama-3-74M	Llama-3-74M/tinyllama-110M/DeepSeek-R1-DRAFT-Qwen2.5-0.5B
User-LLM	Llama-3-74M	Llama-3-74M/tinyllama-110M/DeepSeek-R1-DRAFT-Qwen2.5-0.5B
Item CLIP	None	BGE-VL-base/BGE-VL-large

Table 3. Model sources.

Model	Model Reference Address
Llama-3-74M	https://huggingface.co/KylinC/Llama-3-74M
tinyllama-110M	https://huggingface.co/nickypro/tinyllama-110M
DeepSeek-R1-DRAFT-Qwen2.5-0.5B	https://huggingface.co/alamios/DeepSeek-R1-DRAFT-Qwen2.5-0.5B
TinyLlama-1.1B	https://huggingface.co/aegon-h/TinyLlama-1.1B
BGE-VL-Base	https://huggingface.co/BAAI/BGE-VL-base
BGE-VL-large	https://huggingface.co/BAAI/BGE-VL-large

Note. All URLs in this table were accessed on 22 July 2026.

4.3.1. Experiments with Dataset rec_tmall_60K

For the rec_tmall_60K dataset, performance results for several recommender systems are reported in Tables 4 and 5. We compare the proposed MHLLM with state-of-the-art baselines (see Appendix C.8): Gru4Rec, Bert4Rec, SASRec, and HLLM. In Table 4, MHLLM demonstrates a performance increase in Recall that ranges from 0.2% at Recall@200 to 25.16% at Recall@5 compared to HLLM. Similarly, for NDCG, the performance improvement spans from 11.89% at NDCG@200 to 27.07% at NDCG@5 as shown in Table 5. The detailed configurations of the MHLLM variants MHLLM1, MHLLM2, and MHLLM3 are provided in Appendix C.9.

Table 4. Recall performance of various RSs with BGE-VL-LARGE as CLIP; MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M, and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on rec_tmall_60K.

Model	Recall@5	Recall@10	Recall@50	Recall@200
Gru4Rec	0.00968	0.014821	0.036086	0.065078
Bert4Rec	0.00823	0.012389	0.026128	0.045344
SASRec	0.01772	0.030374	0.045162	0.064295
HLLM	0.09798	0.140248	0.221628	0.231353
MHLLM1	0.1218797	0.1668249	0.2232935	0.23137
MHLLM2	0.168307	0.203843	0.223460	0.227590
MHLLM3	0.135152	0.185492	0.218914	0.222611

Table 5. NDCG performance of various RSs with BGE-VL-LARGE as CLIP; MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M, and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on rec_tmall_60K.

Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
Gru4Rec	0.006071	0.007683	0.012391	0.012391
Bert4Rec	0.005588	0.006943	0.009895	0.012768
SASRec	0.019663	0.021363	0.024652	0.027513
HLLM	0.066560	0.080194	0.098983	0.100508
MHLLM1	0.0859951	0.1009343	0.1133522	0.1146824
MHLLM2	0.1189227	0.1305772	0.1353243	0.1359523
MHLLM3	0.0933856	0.1096649	0.1180721	0.118616

It is worth noting that we also attempted to benchmark against MMSBR [5] and MMGCN [28]; however, due to the absence of their native input modalities (e.g., price hierarchy and acoustic signals), we were obliged to perform feature adaptations. The adapted results, along with a detailed discussion of their inherent limitations, are deferred to Appendix F. Hence, we do not include them in the main comparison to avoid conflating architectural superiority with dataset-specific artifacts.

4.3.2. Experiments with Dataset Pixel200K

To thoroughly evaluate model robustness, we include experiments on a highly sparse benchmark (Pixel200K), where we report both absolute and relative metrics, acknowledging that extreme sparsity inherently depresses absolute Recall values while relative improvements remain informative of model discrimination capacity.

As the original HLLM paper [13] had already conducted comprehensive comparisons proving HLLM superiority over other models like Gru4Rec, Bert4Rec, and SASRec, we would like to see the incremental improvement of MHLLM over HLLM. For the dataset Pixel200K, the results for using various model settings (MHLLM1, MHLLM2, and MHLLM3) are presented in Tables 6 and 7. In Table 6, the MHLLM demonstrates a performance increase in Recall and NDCG.

Table 6. Recall performance of HLLM and MHLLM with MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on Pixel200K.

Model	Recall@5	Recall@10	Recall@50	Recall@200
HLLM	0.000105	0.000185	0.000800	0.002615
MHLLM1	0.000125	0.000200	0.000795	0.003280
MHLLM2	0.000130	0.000200	0.000925	0.003700
MHLLM3	0.000230	0.000375	0.002075	0.006525

Table 7. NDCG performance of HLLM and MHLLM with MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on Pixel200K.

Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
HLLM	0.0000589	0.0000851	0.0002108	0.0004794
MHLLM1	0.0000701	0.0000945	0.0002203	0.0005876
MHLLM2	0.0000810	0.0001033	0.0002567	0.0006649
MHLLM3	0.0001358	0.0001817	0.0005412	0.0011952

For MHLLM1, improvements over the HLLM are notable, with Recall@5, 10, and 200 increasing by 19.0%, 8.1%, and 25.4%, respectively, although there is a slight decrease

at Recall@50. In terms of NDCG, the enhancements are also significant, with increases of 19.0%, 11%, 4.5%, and 22.6% at NDCG@5, 10, 50, and 200, respectively.

MHLLM2 effectively addresses the regression observed in Recall@50 from MHLLM1, resulting in stronger overall performance.

MHLLM3, on the other hand, dramatically surpasses the previous models, including MHLLM1 and MHLLM2, achieving over a 100% performance improvement across all Recall and NDCG metrics.

The low absolute Recall and NDCG values on Pixel200K (e.g., Recall@5 $\approx 1.2 \times 10^{-4}$) result from three factors: (1) extreme data sparsity (5.24 interactions per user); (2) full-item ranking among 96,121 candidates; and (3) the use of a 74M-parameter Llama-3 model with only ~ 1 M training samples. Despite these challenges, MHLLM achieves substantial relative improvements over HLLM (up to 119% in Recall@5 and 131% in NDCG@5), demonstrating the effectiveness of the proposed multimodal approach.

In the MHLLMs, metrics such as Recall@K and NDCG@K show a positive correlation with both the scale of the LLM and the size of the dataset. MHLLM1, MHLLM2, and MHLLM3 are defined as MHLLMs, in which both Item-LLM and User-LLM are simultaneously and respectively set as Llama-3-74M, tinyllama-110M, and DeepSeek-R1-DRAFT-Qwen2.5-0.5B. The parameter sizes for MHLLM1, MHLLM2, and MHLLM3 are 74 MB, 110 MB, and 500 MB, respectively. As the dataset size and model parameters increase, accuracy improves; however, this also leads to higher training costs.

It is important to interpret the results on Pixel200K with caution. The extremely low absolute Recall values (e.g., 1.2×10^{-4} at Recall@5) stem from three factors inherent to this benchmark: (i) extreme data sparsity with only 5.24 interactions per user on average; (ii) the full-item ranking protocol that requires ranking the ground-truth item among 96,121 candidates; and (iii) a limited training sample size of 1,048,576 interactions. Under such challenging conditions, any recommendation model is expected to yield low absolute metrics. However, the relative improvements achieved by MHLLM over the strong HLLM baseline (e.g., +119% in Recall@5 and +131% in NDCG@5) are statistically and methodologically significant. They demonstrate that the proposed multimodal enhancements enable the model to discriminate relevant items more effectively even from extremely noisy and sparse user signals. Nevertheless, from a practical deployment perspective, the absolute gain (e.g., an increase from 0.0105% to 0.0230% in Recall@5) may not translate into substantial business lift in a real-world system. Therefore, while the relative gains validate the architectural superiority of MHLLM, its practical value on such sparse datasets should be further verified on denser interaction data or via online A/B testing.

4.4. Ablation Experiment Results

Ablation tests comparing the HLLM with the Item-LLM and User-LLM—both trained from scratch and those with pre-training—were conducted in [13]. The results show that the system utilizing a pre-trained network consistently outperforms the others across all metrics. Notably, the most significant performance improvement is observed when both the Item-LLM and User-LLM are pre-trained.

Another ablation test compared the HLLM with the Frozen/Learnable setting of the Item-LLM and User-LLM learnable during training. The results show that the learnable Item-LLM and User-LLM networks significantly improve performance across all metrics.

Various sizes of the Item and User models are tested. It is found that for Item mode, TinyLlama, with its larger parameter count (1.1 B), outperforms both BERT models (BERT-Base of 110 M and BERT-Large of 340 M) across all metrics, indicating a strong correlation between model capacity and performance in this context. For the User model, the Recall and NDCG performance comparison of the MHLLM using various LLMs as the User

Model, including SASRec, the Llama model with two hidden decoder layers (denoted as Llama-2L, 100 MB), and TinyLlama-1.1B, was also performed. The User Models are trained from scratch without utilizing pre-trained weights, while all item models employ TinyLlama-1.1B. The results demonstrate that TinyLlama, with 1.1 billion parameters, consistently outperformed both SASRec and Llama-2L across all metrics.

Furthermore, we verify the effectiveness of the core components in MHLLM, including the cross-modal CLIP encoder, dynamic gating mechanism, GELU activation function, and lightweight projection layer, and quantify the contribution of each component to the overall model performance.

4.4.1. Experiments on BGE-VL-Large and BGE-VL-Base Models Based on the CLIP Framework

In this work, we aim to evaluate the performance of using a BGE-VL-base instead of BGE-VL-large models for the CLIP. Tables 8 and 9 present performance metrics on the rec_tmall_60K dataset, while Tables 10 and 11 present metrics on Pixel200K dataset. The results clearly indicate that the larger CLIP models outperform their smaller counterparts.

Table 8. Recall performance of MHLLM with MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on rec_tmall_60K.

With BGE-VL-Large				
Model	Recall@5	Recall@10	Recall@50	Recall@200
MHLLM1	0.1218797	0.1668249	0.2232935	0.23137
MHLLM2	0.168307	0.203843	0.223460	0.227590
MHLLM3	0.135152	0.185492	0.218914	0.222611
With BGE-VL-Base				
Model	Recall@5	Recall@10	Recall@50	Recall@200
MHLLM1	0.100698	0.147991	0.223710	0.234101
MHLLM2	0.104078	0.151122	0.220796	0.233152
MHLLM3	0.078417	0.113220	0.212170	0.234534

Table 9. NDCG performance of MHLLM with MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on rec_tmall_60K.

With BGE-VL-Large				
Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
MHLLM1	0.0838967	0.0984844	0.111944	0.1131951
MHLLM2	0.118923	0.130577	0.135324	0.135952
MHLLM3	0.093386	0.109665	0.118072	0.118626
With BGE-VL-Base				
Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
MHLLM1	0.066791	0.082035	0.099762	0.101342
MHLLM2	0.069468	0.084669	0.101019	0.102882
MHLLM3	0.052759	0.063992	0.086127	0.089674

The experimental results indicate that the parameter size of the CLIP model significantly affects the performance of the MHLLMs. Our proposed MHLLM builds upon the HLLM framework by integrating multimodal CLIP, thereby enhancing recommendation outcomes through the use of both image and text inputs.

Table 10. Recall performance of MHLLM with MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on Pixel200K.

With BGE-VL-Large				
Model	Recall@5	Recall@10	Recall@50	Recall@200
MHLLM1	0.000125	0.000200	0.000795	0.003280
MHLLM2	0.000130	0.000200	0.000925	0.003700
MHLLM3	0.000230	0.000375	0.002075	0.006525
With BGE-VL-Base				
Model	Recall@5	Recall@10	Recall@50	Recall@200
M HLLM1	0.000105	0.000195	0.00100	0.003895
MHLLM2	0.000105	0.000215	0.001005	0.003360
MHLLM3	0.000150	0.000325	0.001455	0.005640

Table 11. NDCG performance of MHLLM with MHLLM1, MHLLM2, and MHLLM3 using Llama-3-74M, tinyllama-110M and DeepSeek-R1-DRAFT-Qwen2.5-0.5B as LLM on Pixel200K.

With BGE-VL-Large				
Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
MHLLM1	0.0000701	0.0000945	0.0002203	0.0005876
MHLLM2	0.0000810	0.0001033	0.0002567	0.0006649
MHLLM3	0.0001358	0.0001817	0.0005412	0.0011952
With BGE-VL-Base				
Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
MHLLM1	0.0000692	0.0000972	0.0002633	0.0006841
MHLLM2	0.0000589	0.0000939	0.0002618	0.0006111
MHLLM3	0.0000905	0.0001497	0.0003896	0.0009992

4.4.2. Experiments on Different Activation Functions in Gated Fusion Mechanism

To verify the influence of the activation function in Equation (6) on the overall model performance within the dynamic gating mechanisms described in Equations (5) and (6), we conduct comparative experiments with four typical activation functions: GELU, ReLU, LeakyReLU, and SiLU. Table 12 presents the Recall results, and Table 13 presents the NDCG results for the MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset.

Table 12. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different activation functions in the fusion control gate.

Activation	Recall@5	Recall@10	Recall@50	Recall@200
GELU	0.1218797	0.1668249	0.2232935	0.23137
ReLU	0.118033	0.1626284	0.2226774	0.2316198
LeakyReLU	0.1169339	0.1615793	0.2222777	0.2304874
Silu	0.1045112	0.1467419	0.2204959	0.2303542

Experiments show that on the rec_tmall_60K dataset, for the MHLLM based on Llama-3-74M, the use of four activation functions (GELU, ReLU, LeakyReLU, and SiLU) in the fusion control gate results in a consistent performance ranking across all Recall (@5/@10/@50/@200) and NDCG (@5/@10/@50/@200) evaluation metrics: GELU > ReLU > LeakyReLU > SiLU. Specifically, GELU achieves the best performance,

outperforming ReLU, LeakyReLU, and SiLU by approximately 3.26%, 4.23%, and 16.62% in Recall@5, respectively, and exceeding SiLU by more than 16% in NDCG@5. In contrast, SiLU performs the worst across all metrics, with a particularly significant decline in head metrics. Additionally, activation functions have a far greater impact on head recommendation metrics such as @5 and @10 than on long-tail metrics like @50 and @200, and the performance gap between different activation functions gradually narrows as the index truncation position moves backward.

Table 13. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different activation functions in the fusion control gate.

Activation	NDCG@5	NDCG@10	NDCG@50	NDCG@200
GELU	0.0838967	0.0984844	0.111944	0.1131951
ReLU	0.0821604	0.0965953	0.1107427	0.1121241
LeakyReLU	0.0794005	0.093868	0.1082132	0.1094891
Silu	0.0719264	0.0855723	0.1026862	0.1042153

4.4.3. Experiments on the Core Components of MHLLM (Detailed in Appendix E)

- **Dynamic gating mechanism** (Appendix E.1): Replacing the proposed gated fusion with a simple HLLM baseline or self-attention fusion significantly degrades performance, confirming the importance of adaptive weighting.
- **Gating smoothing loss** (Appendix E.2): Adding $\mathcal{L}_{GATE}$ yields a ~4.7% relative improvement in Recall@5, confirming its effectiveness in balancing modalities.
- **Lightweight projection layer** (Appendix E.3): Our simple linear mapping + Layer-Norm scheme outperforms a complex 4-layer MLP, achieving an 8% higher Recall@5 with significantly fewer parameters.
- **Task-specific prompts** (Appendix E.4): The proposed prompt is critical. Compared to “No Prompt,” it improves Recall@5 by 26.8%. Declarative and more detailed prompts yield further gains on top-K metrics.
- **[ITEM] token** (Appendix E.5): Using a trainable [ITEM] token instead of mean pooling improves Recall@5 by over 33%, demonstrating its effectiveness in aggregating item semantics.

5. Conclusions

The proposed MHLLM (Multimodal Hierarchical Large Language Model) provides an effective approach for multimodal recommendations, with clear performance improvements over the traditional HLLM. Its innovative dynamic fusion mechanism adjusts the weights of image and text inputs, effectively reducing modal noise and enhancing recommendation accuracy. By utilizing an LLM-based deep encoder with attention pooling, the MHLLM captures complex user interaction patterns for more personalized recommendations. A step-by-step training strategy is implemented, breaking down components like the Item-LLM and User-LLM for pre-training. Additionally, a feature caching mechanism minimizes overhead, while asynchronous preprocessing ensures efficient, low-latency recommendations. Future research will focus on lightweight technologies and validation on larger-scale benchmarks, while the current model’s strong discriminative ability is already evidenced by up to 119% relative Recall@5 and +131% in NDCG@5 gain on the highly sparse Pixel200K dataset, further demonstrating its robust discriminative capability under challenging data conditions.

Supplementary Materials: The codes for generating the simulation results reported in the paper are available in <https://github.com/rebingzi/MHLLM> (accessed on 22 July 2026).

Author Contributions: Conceptualization, W.-L.D. and R.T.-C.H.; methodology, W.-L.D. and R.T.-C.H.; software, W.-L.D. and R.T.-C.H.; validation, W.-L.D., R.T.-C.H., H.S.-H.T. and T.Y.Z.; formal analysis, W.-L.D., R.T.-C.H., H.S.-H.T., T.Y.Z. and W.-L.L.; investigation, W.-L.D. and R.T.-C.H.; resources, W.-L.D., R.T.-C.H., H.S.-H.T., T.Y.Z. and W.-L.L.; data curation, W.-L.D.; writing—original draft preparation, W.-L.D. and R.T.-C.H.; writing—review and editing, W.-L.D., R.T.-C.H., H.S.-H.T., T.Y.Z. and W.-L.L.; visualization, W.-L.D.; supervision, R.T.-C.H. and W.-L.L.; project administration, R.T.-C.H. and W.-L.L.; funding acquisition, R.T.-C.H., H.S.-H.T., T.Y.Z. and W.-L.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original data presented in this study are openly available from two public sources. The Alibaba Tianchi dataset is available at <https://tianchi.aliyun.com/dataset/140281> (accessed on 22 July 2026), and the Pixel200K dataset is available at <https://github.com/westlake-repl/PixelRec> (accessed on 22 July 2026). The derived dataset rec_tmall_60K generated from the Alibaba Tianchi data and analyzed during this study is available from the corresponding author upon reasonable request. The Pixel200K dataset was used in its original form.

Acknowledgments: The work described in this paper was fully supported by the Hong Kong Chu Hai College. During the preparation of this manuscript, the authors used some free generative AI solely to refine and improve the grammar of the text. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CBF	Content-Based Filtering
CF	Collaborative Filtering
CIR	Combined Image Retrieval
CLIP	Contrastive Language-Image Pre-training
CNN	Convolutional neural networks
CTR	Click-Through Rate
CVR	Conversion Rate
DIN	Deep Interest Network
DNN	Deep neural networks
ESMM	Entire Space Multi-task Model
GNN	Graph Neural Networks
HLLM	Hierarchical large language model
LSM	Latent semantic models
LLM	Large language model
LLM4Rec	Large Language Model for Recommendation
MF	Matrix factorization
MHLLM	Multimodal hierarchical large language model
MMoE	Multi-gate Mixture of Experts
MMRet	Multimodal retrieval
MMSBR	Multimodal Session-Based Recommendation
MLP	Multi-layer perceptron
NDCG	Normalized Discounted Cumulative Gain

RS	Recommender system
SASRec	Self-Attention recommendation algorithm
SFT	Supervised fine-tuning

Appendix A. Semantic Alignment

A cross-modal feature encoder is constructed using CLIP for the semantic alignment between text and images:

Text encoding process

- Use AutoTokenizer to dynamically truncate Chinese text (the single-word width is 3, and the maximum sequence length is calculated based on the actual character width).
- The truncated text is input into the CLIP text encoder to generate 768-dimensional features.
- The features are mapped to the model hidden layer (such as 4096 dimensions for Llama) through the text_proj layer (Linear + LayerNorm).

Image encoding optimization

- Dual-mode support:** supports pre-extraction of features (directly loading image_features) and real-time encoding (processing image paths through clip_processor).
- Parameter freezing:** By default, CLIP visual encoder parameters are frozen (freeze_clip = True), and only the projection layer is optimized.
- Multi-GPU acceleration:** CLIP half-precision (FP16) parallel encoding is implemented through nn.DataParallel, and the throughput is improved.

Appendix B. LLM Fine-Tuning

Appendix B.1. Supervised Fine-Tuning (SFT) of LLM

Guided by key indicators of the recommendation system, such as Recall, we leverage user–product interaction data—including click, favorite, and purchase records—from e-commerce scenarios to conduct supervised fine-tuning on a pre-trained LLM like TinyLlama. The fine-tuning data is organized in a tuple format: (user history sequence, current query, positive sample product, negative sample product). This structure effectively facilitates the model’s ability to learn the mapping relationship between user interests and product features.

Input and output reconstruction:

Input: Fusion feature sequence of user’s historical interaction items $\{E_1, E_2 \dots, E_n\}$ with the current query features E_{query} (text or mixed text and image features);

Output: Predicted product features E_{pred} ; the similarity with positive/negative sample features is measured by contrastive loss:

$$\mathcal{L}_{llm} = -\log \frac{\exp\left(\frac{\text{Sim}(E_{\text{pred}}, E_{\text{pos}})}{\tau}\right)}{\exp\left(\frac{\text{Sim}(E_{\text{pred}}, E_{\text{pos}})}{\tau}\right) + \sum_{k=1}^K \exp\left(\frac{\text{Sim}(E_{\text{pred}}, E_{\text{neg}_k})}{\tau}\right)} \quad (\text{A1})$$

where τ is the temperature coefficient and K is the number of negative samples.

Appendix B.2. Fine-Tuning the CLIP Model for E-Commerce Scenarios

The CLIP model performs well in general image–text alignment tasks, but there is a domain gap when it is directly applied to e-commerce scenarios (such as product image–title matching). This study uses the following strategies for optimization:

(1) Pre-training weight initialization

The initial parameters are CLIP weights (such as the ViT-B/32 architecture) [31] pre-trained on the MegaPairs dataset, which contains large-scale cross-lingual image–text pairs and has strong semantic generalization capabilities.

(2) Supervised Fine-tuning (SFT) Process

- (a) **Data construction:** Collect product image and text data (such as titles, detail page text, and product images) from e-commerce platforms. Construct aligned pairs of positive and negative samples: positive samples consist of image and text combinations for the same product, while negative samples feature random pairs of images and texts from different products.
- (b) **Objective function:** Image–text contrastive loss and text-to-image generation loss are jointly optimized using the following objective function:

$$\mathcal{L}_{clips} = \mathcal{L}_{ITC} + \lambda \mathcal{L}_{T2I} \quad (\text{A2})$$

where $\mathcal{L}_{ITC}$ enforces semantic similarity between images and texts, while $\mathcal{L}_{T2I}$ (text to image) ensure that the text accurately generates the feature representation of the corresponding image. The parameter λ is the balance coefficient.

- (c) **End-to-end training:** CLIP and LLM modules are jointly trained to achieve collaborative optimization of image and text feature extraction and user interest modeling. Experiments show that this strategy can improve the accuracy of product image and text alignment, and the Recall rate of cross-modal recommendation is also significantly improved.

Appendix B.3. Training Strategies and Performance Improvement

The training process follows a hierarchical structure, consisting of the following steps:

- (1) **Pre-training stage:** LLM is pre-trained based on general text corpus, and CLIP is pre-trained based on MegaPairs.
- (2) **Fine-tuning stage:** First, fine-tune CLIP independently until the image–text alignment converges, then freeze the CLIP weights and fine-tune the LLM and fusion modules (such as Gate, Fusion), and finally perform end-to-end joint fine-tuning to avoid gradient vanishing and improve training stability.

RSs are generally classified into two categories: generative recommendations and discriminative recommendations. The MHLLM can seamlessly adapt to both types by flexibly adjusting its training objectives. For generative recommendations, the primary goal is to predict the next item. Based on the fusion features of the user's historical interaction sequence $\{E_1, E_2, \dots, E_n\}$, it generates the product features E_i' that may be interacted with in the future through User-LLM. The training process employs the InfoNCE loss function to ensure that the features of positive samples are closely aligned in the embedding space, while negative sample features are kept at a greater distance. The specific formula [13] is

$$\mathcal{L}_{gen} = - \sum_{j=1}^b \sum_{i=2}^n \log \frac{e^{s(E'_{j,i}, E_{j,i})}}{e^{s(E'_{j,i}, E_{j,i})} + \sum_{\gamma}^N e^{s(E'_{j,i}, E_{j,i,k})}} \quad (\text{A3})$$

- $s(\cdot)$: cosine similarity function with temperature parameter τ .
- $E_{j,i}$: The fusion feature of the i -th item in the j -th user history (Item-LLM + CLIP).
- $E'_{j,i}$: User-LLM predicts the i -th product feature.
- $E_{j,i,k}$: Randomly sampled negative sample features, N is the number of negative samples.
- b : batch size, n : history sequence length.

Appendix C. MHLLM Development Process

Appendix C.1. Pre-Training and Target Fine-Tuning of Item-LLM and User-LLM

According to the research results of the HLLM model, pre-training and task fine-tuning are the core links to improve the recommendation performance of the language model (LLM). This project adopts the following strategies:

(a) **Pre-training weight selection:**

Item-LLM: Based on the tinyllama-110M model, it uses e-commerce product text data (such as titles and detailed descriptions) for pre-training to capture the semantic features of products.

User-LLM: Use the tinyllama-110M model, pre-trained based on user interaction logs (such as query clicks, purchase sequences), to model the dynamic changes of user interests.

(b) **Fine-tuning target:**

The two types of LLM are fine-tuned through recommendation tasks, and the optimization directions are as follows: enhance the semantic alignment between product text features and user interests, and improve the model's generalization ability for long-tail products and sparse interaction data.

(c) **Experimental conclusion:**

As the LLM parameter scale increases (e.g., from 1 billion to 3.2 billion parameters), the Recall@50 metric of the recommendation task improves by 12–18%, verifying the positive correlation between the pre-training data scale and the recommendation performance.

Appendix C.2. E-Commerce Scenario-Specific Fine-Tuning of the CLIP Model

The CLIP model pre-trained with the MegaPairs dataset is selected as the basis and optimized for e-commerce image and text data:

(a) **Weight freezing and adapter training:**

The core weights of the CLIP visual encoder and text encoder are fixed, and only the projection layer (Text Project/Image Project) is fine-tuned.

The feature dimension (768 dimensions) of CLIP output is aligned to the hidden layer dimension (e.g., 1024 dimensions) of Item-LLM through the projection layer to ensure the compatibility of multimodal features.

(b) **Loss function design:**

CLIP InfoNCE Loss

$$\mathcal{L}_{clip} = -\frac{1}{2N} \sum_{i=1}^N \left[\log \frac{e^{s \cdot \text{sim}(t_i, v_j)}}{\sum_{j=1}^N e^{s \cdot \text{sim}(t_i, v_j)}} + \log \frac{e^{s \cdot \text{sim}(v_i, t_j)}}{\sum_{j=1}^N e^{s \cdot \text{sim}(v_i, t_j)}} \right] \quad (\text{A4})$$

s is the temperature parameter (logit_scale), which is used to dynamically adjust the distribution of the feature space.

$\text{sim}(t, v)$ represents the cosine similarity between text feature t and image feature v .

CLIP Alignment Loss:

$$\mathcal{L}_{align} = \frac{1}{2N} \sum_{i=1}^N \left[\mathcal{L}_{text2image}^{(i)} + \mathcal{L}_{image2text}^{(i)} \right] \quad (\text{A5})$$

This loss function forces the image and text features generated by CLIP to be close to the text features of Item-LLM in the embedding space, thereby improving the consistency of cross-modal semantics.

Appendix C.3. E-Commerce Data Preprocessing and Feature Caching

The CLIP model pre-trained with the MegaPairs dataset is selected as the basis and optimized for e-commerce image and text data:

(a) **Weight freezing and adapter training:**

The core weights of the CLIP visual encoder and text encoder are fixed, and only the pre-trained CLIP model is used to perform offline feature extraction on the entire e-commerce data and build a cache library for efficient retrieval:

(b) **Text feature cache:**

Input: Product title text, generate text embedding vector through CLIP text encoder Emb_{text} . **Storage:** Use product ID as key and feature vector as value, and save to text_cache_features.bin file.

(c) **Image feature cache:**

Input: Product image, image embedding vector generated by CLIP visual encoder Emb_{image} . **Storage:** Use product ID as key and feature vector as value and save to image_cache_features.bin file.

(d) **Performance optimization:**

There is no need to repeatedly call the CLIP encoder during the training phase, reducing computing power consumption by 30%. The feature retrieval delay is less than 10 milliseconds, which significantly improves the training efficiency.

Appendix C.4. End-to-End Training of MHLLM

(a) **Model integration process:**

Load the pre-trained Item-LLM, User-LLM and CLIP model parameters. Read the graphic and text features of the product from the cache and fuse them with the text features output by Item-LLM through a gating mechanism.

Input the user's historical interaction sequence into User-LLM, predict the characteristics of the next product, and calculate the comprehensive loss:

$$\mathcal{L}_{total} = \mathcal{L}_{llm} + \lambda \mathcal{L}_{clip} + \mu \mathcal{L}_{align} \quad (A6)$$

Among them, λ and μ are weight parameters used to balance the LLM prediction error and CLIP alignment error.

(b) **Optimization strategy:**

The AdamW optimizer is used, and the learning rate adopts a step-wise decay strategy (the initial value is 1×10^{-4} , and it decays by 0.8 times every five training cycles). Enable Gradient Checkpointing technology to reduce video memory usage by 50%.

Appendix C.5. Cross-Modal Retrieval and Recommendation Prediction

The MHLLM supports two core application scenarios:

(a) **No-query recommendation:**

Generate a list of recommended products based only on the user's historical interaction sequence through User-LLM.

(b) **Image and text search:**

The user enters an image or text query, and the CLIP model extracts query features Emb_{query} . User-LLM generates interest features based on user history sequences Emb_{user} . The query features and user interest features are fused by element-by-element multiplication:

$$Emb_{pred} = Emb_{query} \odot Emb_{user} \quad (A7)$$

Then, the cosine similarity is sorted with the product features in the cache, and the search results are returned.

Appendix C.6. Elaboration on Implicit User Behavior Signals in Section 3.2.4(a)

Section 3.2.4(a) describes the user interest modeling logic of User-LLM. Here, “implicit user behavior signals” are supplementary features correlated with user interests to improve modeling accuracy, excluding the fused e_{Item} output from the Item module. Their definitions, categories, and fusion methods are standardized, as detailed below.

(a) **Construction:**

“implicit user behavior signals” are non-modal features derived from user–item behaviors and attributes. They are independent of the semantic embeddings produced by Item-CLIP and Item-LLM, and they supplement semantic signals by better capturing users’ behavioral preferences.

(b) **Categories and extraction (e-commerce setting):**

Behavior features: normalized counts of clicks, collections, cart adds, and purchases for the item’s category $\rightarrow f_{beh} \in \mathbb{R}^4$ (mapped to $[0, 1]$).

Temporal features: latest interaction time and time interval since the last interaction $\rightarrow f_{time} \in \mathbb{R}^d$ via linear mapping to match e_{Item} ’s dimension.

Attribute matching: cosine similarity between user attributes (e.g., age, gender, consumption level) and item attributes (e.g., category, price range, style) $\rightarrow f_{match} \in [0, 1]$, then expanded to $\mathbb{R}^d$.

(c) **Input to User-LLM:**

Concatenate $f_{beh}', f_{time}', f_{match}'$ (after dimension expansion of f_{match}', f_{beh}') with e_{Item} to feed into User-LLM: $f_{user_input} = [e_{Item}; f_{beh}'; f_{time}'; f_{match}']$. This ensures the synergistic effect of “implicit user behavior signals” and semantic embedding features.

Appendix C.7. Detailed Implementation of Feature Dimension Projection Alignment in MHLLM

Feature dimension projection alignment serves as the core module of the cross-modal fusion pipeline in MHLLM. Two mainstream implementation schemes are available for this critical component: the lightweight projection layer proposed in the original MHLLM paper, and the traditional projection layer integrating multi-round feature transformation and complex normalization. This appendix elaborates the detailed code implementation and inherent design logic of the traditional projection layer and systematically analyzes the core discrepancies between the two schemes in design philosophy and computational efficiency through a multi-dimensional comparative analysis with the original lightweight scheme.

(1) **Traditional Projection Layer Scheme (Multi-Round Feature Transformation + Complex Normalization)**

The traditional projection layer is constructed based on multi-round linear feature transformation and integrates advanced design strategies including interval residual connections, hybrid normalization, adaptive activation, and regularization mechanisms. The core implementation logic and detailed design specifications are elaborated as follows.

(a) **Core Logic of Class Initialization**

The initialization phase constructs a 4-layer feature transformation network and simultaneously initializes activation functions, regularization modules, residual adapters and the final normalization layer. The core design principles are outlined below:

- (i) **Basic Component Configuration:** A fixed 4-layer feature transformation structure is deployed (consistent with the conventional 3–4-layer design in classical multimodal fusion works), equipped with the GELU activation function and Dropout (0.2) for model regularization.
- (ii) **Linear Layer Dimension Mapping:** The input layer projects input features to the preset hidden dimension, intermediate layers maintain consistent hidden dimensions for stable feature learning, and the output layer maps hidden features to the target unified dimension; all linear layers are configured with learnable bias terms to enhance feature fitting capability.
- (iii) **Hybrid Normalization Mechanism:** The output layer solely adopts LayerNorm for final feature calibration, while all other hidden layers employ a LayerNorm plus interval BatchNorm1d strategy (an extra BatchNorm1d is added to even-numbered layers), and all normalization layers are equipped with learnable scale and bias parameters.
- (iv) **Residual Adapter Design:** A dedicated linear adapter is deployed for dimension alignment when input and output dimensions mismatch; for consistent dimensions, identity mapping is applied to ensure valid and stable residual connections.
- (v) **Layer Encapsulation and Management:** Each transformation layer is sequentially encapsulated with a linear layer, activation function, normalization module and Dropout layer, and all sub-modules are uniformly managed via PyTorch ModuleList for standardized deployment.

(b) **Core Process of Forward Propagation**

The forward propagation pipeline executes multi-round feature transformation and residual feature fusion and outputs cross-modal features with aligned dimensions following the standardized workflow below:

- (i) **Residual Initialization:** Raw input features are processed by the residual adapter to generate compatible residual features for subsequent final fusion.
- (ii) **Feature Flattening:** Input feature tensors are flattened into 2D matrices to match the computational dimension requirements of fully connected linear layers.
- (iii) **Multi-Round Transformation with Interval Residuals:** The 4-layer sub-modules are traversed sequentially, and interval residual connections ($x = x + \text{preceding layer features}$) are implemented after feature transformation in odd-numbered layers on the premise of consistent feature dimensions.
- (iv) **Tensor Shape Restoration:** Flattened features are reshaped back to the original [Batch Size B, Sequence Length L, Feature Dimension D] tensor format to maintain sequence-level feature integrity.
- (v) **Final Fusion and Normalization:** The multi-round transformed features are fused with the initialized residual features via element-wise addition, and the final aligned features are output after standardized LayerNorm calibration.

- (2) **Core Design Comparison Between Traditional Scheme and Original Lightweight Scheme**
The original lightweight projection layer, as proposed in the official MHLLM paper, adopts a simplified single linear mapping architecture, which differs drastically from the traditional multi-round feature transformation scheme in core design. The detailed multi-dimensional comparison between the two schemes is presented in the following Table A1:

Table A1. Comparison of traditional scheme and original lightweight scheme in projection layer.

Comparison Dimension	Original Lightweight Projection Layer (Paper Scheme)	Traditional Multi-Round Feature Transformation Scheme
Network Layers	1 linear layer	4 linear layers (3–4 layers conform to traditional scheme design)
Bias Term	No bias (Linear(..., bias = False))	With bias (Linear(..., bias = True))
Normalization	Simplified LayerNorm (without redundant bias)	LayerNorm with scale/bias + Interval BatchNorm1d
Activation Function	None	GELU activation (executed after each feature transformation layer)
Regularization	None	Dropout(0.2) (executed after each normalization layer)
Feature Transformation Mode	Single-round direct linear mapping	Multi-round linear transformation + Interval residual connection (once every 2 layers)
Intermediate Dimension	No intermediate dimension, direct mapping to target dimension	Doubled intermediate dimension (hidden_size $\times$ 2)

(3) **Core Computational Differences Between the Two Schemes**

Driven by distinct design philosophies, the traditional scheme exhibits substantial advantages in feature representation capability but brings corresponding increments in parameter scale, computational complexity and memory footprint compared with the original lightweight scheme. The core computational differences are summarized as follows:

- (a) **Parameter Scale:** The traditional scheme consists of four stacked linear layers (with doubled hidden intermediate dimensions) and multiple hybrid normalization modules, resulting in an overall parameter count that is 8 to 10 times that of the original lightweight projection layer.
- (b) **Computational Complexity:** The original lightweight scheme only involves one matrix multiplication and one single LayerNorm operation, whereas the traditional scheme requires 4 matrix multiplications, 4 GELU activation operations, 4 normalization steps and 2 residual fusions, lifting the overall computational load by approximately 60%.
- (c) **Memory Occupancy:** Owing to the doubled intermediate feature dimension and the need to cache intermediate feature tensors during multi-round transformation, the GPU memory occupancy of the traditional scheme is roughly 50% higher than that of the original lightweight scheme.

The experimental results for the above two feature dimension mapping alignment methods are presented in Appendix E.3 (Tables A10 and A11).

Appendix C.8. Comparative Experimental Design

The CLIP model pre-trained with the MegaPairs dataset is selected as the basis. In order to verify the performance advantage of the MHLLM, the following baseline models are selected for comparison:

SASRec: A Transformer-based sequential recommendation model that only processes text interaction data [12].

GRU4Rec: A temporal recommendation model based on the gated recurrent unit (GRU), ignoring visual features [24].

Bert4Rec: A sequence recommendation model based on the bidirectional encoder of Transformer (BERT), which learns the context-dependent relationships of user interaction sequences through masked prediction tasks to improve recommendation performance. [17].

HLLM: A pure language model recommendation system without introducing image modality.

Evaluation metrics: Recall, NDCG, the range is [5, 10, 50, 200], inference latency (milliseconds);

Dataset: Pixel200K, Pixel1M, Pixel8M and Tmall datasets.

Appendix C.9. Definition of MHLLM Variants

Table A2. Definition of MHLLM variants.

Modules	Item-LLM	Item-CLIP	User-LLM
MHLLM1	Llama-3-74M	BGE-VL-large	Llama-3-74M
MHLLM2	tinyllama-110M	BGE-VL-large	tinyllama-110M
MHLLM3	DeepSeek-R1-DRAFT-Qwen2.5-0.5B	BGE-VL-large	DeepSeek-R1-DRAFT-Qwen2.5-0.5B

Appendix D. Characteristics of MHLLM and Comparison with Other Models

Table A3. General characteristics.

Modules	HLLM	MHLLM
Multimodal support	None. Only text-related input and calculation are processed	Yes. The CLIP model extracts features from text and images and performs feature fusion via a multimodal projection layer and fusion gating mechanism to support multimodal input
Item Embed	LLM generation only	LLM + Multimodal CLIP Fusion
Loss Function	Single contrast loss	Contrastive loss + alignment loss
Data Flow	Plain text input	Text + image multimodal input
Function	Recommended generation only	Image and text retrieval + recommendation generation fusion
Modules	HLLM	MHLLM

Table A4. Key Component Description.

Components	Technical Implementation	Performance Optimization Points
CLIP Encoder	Text encoding: BERT variants Image encoding: ViT ResNet	FP16 mixed precision Asynchronous batch processing
Feature Fusion Gate	Dynamic weight calculation: gate = $\sigma(W \cdot [\text{text}; \text{image}] + b)$	Matrix parallel computing

Table A4. *Cont.*

Components	Technical Implementation	Performance Optimization Points
Sequence Encoder	LLaMA-like architecture	Flash Attention Optimization Gradient checkpointing
Feature Cache	LRU strategy + hash index	Sharded storage + quantization compression

Table A5. Module function comparison.

Modules	Input	Output	Technologies
CLIP Double Encoding	Original text/image	768-dimensional features	ViT/BERT Architecture
Feature Projection	CLIP Features	hidden dimension vector	LayerNorm linear transformation
Gated Fusion	Bimodal features	Fusion Features	Adaptive Weight Gating
LLM Encoder	Historical sequence	User Characterization	FlashAttention Optimization
Contrastive Learning	Positive and negative samples	Loss value	Temperature scaled sampling

Appendix E. Detailed Ablation Study Results

Appendix E.1. Effect of Different Fusion Mechanisms

To validate the dynamic gate mechanism's superiority, we conducted two controlled ablation experiments. First, we removed the gated fusion module, retaining only the LLM-Item feature representation as the "HLLM baseline." Second, we replaced "Gated fusion" with the traditional "Self-Attention fusion" while keeping other model parameters unchanged. The results are presented below.

Table A6. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different fusion control mechanisms.

Fusion Mechanism	Recall@5	Recall@10	Recall@50	Recall@200
HLLM baseline	0.09798	0.140248	0.221628	0.231353
Gated fusion	0.1218797	0.1668249	0.2232935	0.23137
Self-Attention fusion	8.33×10^{-5}	0.0002331	0.0014821	0.0042131

Table A7. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different fusion control mechanisms.

Fusion Mechanism	NDCG@5	NDCG@10	NDCG@50	NDCG@200
HLLM baseline	0.066560	0.080194	0.098983	0.100508
Gated fusion	0.0838967	0.0984844	0.111944	0.1131951
Self-Attention fusion	3.89×10^{-5}	8.72×10^{-5}	0.0003433	0.0007449

Experimental results demonstrate that the dynamic gating mechanism outperforms all evaluation metrics, significantly exceeding the baseline performance of HLLM baseline and outperforming the traditional Self-Attention Fusion mechanism, which performs poorly in this recommendation task.

Appendix E.2. Experiments on Gating Coefficients Are Introduced to Construct the Joint Loss Function

To further constrain the dispersion of gating coefficients, avoid extreme weight distribution (e.g., the gating coefficient g_{LLM} is approximately 0 or 1), and ensure that both

dual-modal features can effectively participate in the fusion process, a loss term related to gating coefficients is introduced to construct the joint loss function. The final total joint loss function is defined as $L_{Total} = L_{Rec} + \lambda_1 L_{Align} + \lambda_2 L_{GATE}$, where L_{GATE} is the newly added gating smoothing loss. This gating smoothing loss is used to measure the dispersion of gating coefficients, and its expression is $L_{GATE} = \frac{1}{N} \sum_{i=1}^N \text{Var}(g_{LLM}^i)$, where N denotes the total number of samples, and g_{LLM}^i represents the gating coefficient vector corresponding to the i -th sample. The experimental data with and without the gating smoothing loss will be compared and analyzed in the following section to verify the effectiveness of this loss term.

Table A8. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different gate loss mechanisms.

Loss Mechanism	Recall@5	Recall@10	Recall@50	Recall@200
No L_{GATE}	0.1218797	0.1668249	0.2232935	0.23137
L_{GATE}	0.1308721	0.1742186	0.2238597	0.2315698

Table A9. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different gate loss mechanism.

Loss Mechanism	NDCG@5	NDCG@10	NDCG@50	NDCG@200
No L_{GATE}	0.0838967	0.0984844	0.111944	0.1131951
L_{GATE}	0.0910249	0.1050861	0.1169097	0.1180883

Experimental results show that L_{GATE} achieves a slight improvement of approximately 4.7%, 1.8%, and 0.3% in Recall@5, Recall@10, and Recall@50, respectively, but a slight decrease in Recall@200. In terms of NDCG metrics, L_{GATE} performs comprehensively better than No L_{GATE} , with the improvement rates of each sub-metric ranging from 3.0% to 5.8%, among which the ranking accuracy within the Top-10 range is the most significantly improved. In summary, L_{GATE} can generally optimize the recommendation performance of the MHLLM, verifying its effectiveness.

Appendix E.3. Experimental Study on Feature Dimension Projection Alignment Based on Various Mapping Strategies

To verify the efficiency optimization of feature dimension projection alignment for the proposed model, this study adopts a lightweight implementation scheme of linear mapping + LayerNorm(LM-LN) for the dedicated projection layer (Proj($\cdot$)) of the MHLLM. A comparative experiment is conducted between this scheme and the conventional scheme in cross-modal fusion, which consists of **four** linear layers (including intermediate dimension doubling) + multiple normalization layers(4LL-DD-Norm) (details are provided in Appendix C.7). The experimental data are presented as follows:

Table A10. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different mapping strategies.

Mapping Strategies	Recall@5	Recall@10	Recall@50	Recall@200
LM-LN	0.1218797	0.1668249	0.2232935	0.23137
4LL-DD-Norm	0.1128208	0.156034	0.2217115	0.2306539

Table A11. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different mapping strategies.

Mapping Strategies	NDCG@5	NDCG@10	NDCG@50	NDCG@200
LM-LN	0.0838967	0.0984844	0.111944	0.1131951
4LL-DD-Norm	0.0780877	0.092062	0.1074535	0.1088444

The results demonstrate that the lightweight LM-LN scheme outperforms the traditional complex scheme across all eight metrics, including Recall@5/10/50/200 and NDCG@5/10/50/200. Specifically, the top-tier core indicators Recall@5 and NDCG@5 are improved by 8.03% and 7.44%, respectively, while the long-tail metrics also achieve steady and moderate gains. Adopting only a single-layer linear projection and a single-layer normalization structure, the proposed scheme has significantly fewer parameters and lower computational complexity than conventional approaches. While drastically reducing computational overhead and achieving efficiency optimization, the method incurs no performance degradation; instead, it notably enhances both sample coverage and top-ranking precision in cross-modal retrieval. This sufficiently validates that concise projection alignment architectures are better suited for the MHLLM cross-modal fusion framework, verifying the effectiveness and practicality of this lightweight optimization strategy.

Appendix E.4. Ablation Experiments on Prompts

Prompts act as a task-specific prefix conditioning mechanism for autoregressive large language models, formally reorienting them from open-ended text generation to targeted semantic compression and global item feature extraction. This conditioning helps aggregate product semantics into unified embeddings, as supported by representation learning principles. The design of prompts directly determines the quality of item semantic representations and their alignment with downstream recommendation tasks, making it a key factor affecting multimodal recommendation performance.

To objectively and rigorously verify the role of prompt engineering in the MHLLM framework, we conduct a set of controlled ablation experiments. All experimental settings are kept strictly consistent, including model architecture, training strategy, optimization configuration, and feature extraction pipeline; only the prompt design is varied. Four typical prompt schemes are compared:

- No prompt:** No guiding instruction is added to the model input. That is, only the input data is fed into the LLM and let it generates some output.
- Generic prompt:** A general-purpose instruction without task-specific adaptation. “Extract the item feature:” is used as prompt.
- Our prompt:** The dedicated task-specific prompt designed in this work for e-commerce item feature extraction (Table 1).
- Random prompt:** A randomly constructed prompt, e.g., “Calculate the number of vowels in this item”.

This controlled comparison allows us to quantitatively validate the necessity, effectiveness, and superiority of the task-specific prompt proposed in this paper.

Table A12. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different prompts.

Prompts	Recall@5	Recall@10	Recall@50	Recall@200
Our Prompt	0.1218797	0.1668249	0.2232935	0.23137
No Prompt	0.0961183	0.1394814	0.2176317	0.2289554

Table A12. *Cont.*

Prompts	Recall@5	Recall@10	Recall@50	Recall@200
General prompt	0.1045944	0.1484238	0.2220113	0.2319195
Random prompt	0.087459	0.1266923	0.2151838	0.2294383

Table A13. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different prompts.

Prompts	NDCG@5	NDCG@10	NDCG@50	NDCG@200
Our Prompt	0.0838967	0.0984844	0.111944	0.1131951
No Prompt	0.0645521	0.0785757	0.0966511	0.0984411
General prompt	0.0713301	0.0854885	0.1026371	0.1041719
Random prompt	0.0587924	0.0714283	0.0916544	0.0939153

Experimental results demonstrate a consistent ranking across most key metrics, especially top-k recommendation indicators: Our Prompt > General prompt > No Prompt > Random prompt.

Specifically, Recall@200 of the general prompt is slightly higher than that of our prompt, while our prompt achieves clear advantages in all other metrics.

Compared with the no-prompt baseline, our task-specific prompt yields substantial gains: 26.8% improvement in Recall@5 and 30.0% improvement in NDCG@5.

Notably, prompts exert a much stronger influence on top-k ranking metrics (e.g., Top-5 and Top-10) than on long-tail metrics (Top-50, Top-200). This observation confirms that well-designed task-specific prompts effectively steer the model to produce high-fidelity item semantic embeddings, which in turn significantly improves the precision of high-priority recommendations.

To further explore the impact of task guidance strength on the performance of the MHLLM framework in e-commerce item feature extraction and downstream recommendation tasks, we conduct an additional set of controlled ablation experiments. Based on the aforementioned experimental settings (Llama-3-74M as the LLM, BGE-VL-Large as the CLIP model, and rec_tmall_60K dataset), we fix the prompt to be task-specific (consistent with the core objective of e-commerce recommendation) and only adjust the level of guidance detail. Three typical prompt schemes are designed with clear differences in semantic richness and task guidance specificity:

- (a) **Concise prompt:** “Extract item embedding.” This prompt is the most concise and only conveys the core operation requirement, without additional task context or constraints.
- (b) **Our prompt:** Compress the following sentence into embedding. Based on the concise prompt, this version adds explicit guidance for “sentence compression” and provides basic context to help the model understand the input processing objective.
- (c) **Detailed prompt:** “Compress the product information (title, category, brand) into a dense embedding for e-commerce recommendation.” This prompt provides the most comprehensive task guidance by clearly specifying the input content (product title, category, brand), output form (dense embedding), and application scenario (e-commerce recommendation), enabling the model to better align with the target recommendation task.

Table A14. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using prompts with different guidance strength.

Prompts	Recall@5	Recall@10	Recall@50	Recall@200
Concise Prompt	0.1045944	0.1484238	0.2220113	0.2319195
Our Prompt	0.1218797	0.1668249	0.2232935	0.23137
Detailed prompt	0.1536694	0.1927362	0.2229771	0.2279396

Table A15. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using prompts with different guidance strength.

Prompts	NDCG@5	NDCG@10	NDCG@50	NDCG@200
Concise Prompt	0.0713301	0.0854885	0.1026371	0.1041719
Our Prompt	0.0838967	0.0984844	0.111944	0.1131951
Detailed prompt	0.1097118	0.1224598	0.1297917	0.1305527

The experimental results show that task guidance strength and semantic completeness have a significant positive impact on the e-commerce feature extraction performance of MHLLM within an appropriate range: prompts with clearer task constraints and richer guidance can better direct the model to generate high-quality item embeddings for e-commerce recommendations.

Concise prompts, with insufficient guidance information, fail to fully lead the model to understand the task objective, resulting in coarse feature representation and the lowest Recall and ranking performance.

Standard prompts act as a transitional scheme; their basic operational guidance steadily improves performance over concise prompts.

Detailed prompts achieve the most significant performance gains on low k-value metrics (@5/@10), which aligns better with the core requirement of accurate top-k e-commerce recommendation.

Although the performance gain of detailed prompts tends to saturate or slightly decrease at high k-values (e.g., Recall@200), the NDCG metrics still maintain steady improvement.

Practical implication: For e-commerce item feature extraction and recommendation, MHLLM is recommended to employ task-specific prompts with sufficient guidance detail that clearly define input, output, and application scenarios, as they can significantly improve top-k recommendation performance.

On the basis of exploring prompt type and length, we further investigate the impact of prompt sentence type (imperative, declarative, interrogative) on the performance of the MHLLM framework. Still adhering to the consistent experimental settings (Llama-3-74M as the LLM, BGE-VL-Large as the CLIP model, rec_tmall_60K dataset), we fix the prompt length as medium (to balance guidance clarity and conciseness) and task-specific attribute (focused on e-commerce item embedding extraction), only changing the sentence type of the prompt. Three typical prompt sentence type schemes are designed with distinct differences in expression style and guidance logic:

- (a) **Our prompt:** “Compress the following sentence into embedding: [Product text information]”. This prompt directly issues an operational instruction, guiding the model to complete the embedding extraction task in a straightforward manner, with strong directivity and conciseness.
- (b) **Declarative prompt:** “This is an item embedding extraction task; please process the following product information into a dense embedding: [Product text information]”. This prompt first clarifies the task nature, then puts forward the operational require-

ments, providing a clear task context for the model and reducing the ambiguity of task understanding.

- (c) **Interrogative prompt:** “What is the dense embedding of the following product information for e-commerce recommendation? [Product text information]”. This prompt guides the model to complete the embedding extraction task in the form of a question, which may stimulate the model’s reasoning ability and improve the attention to task details.

The purpose of this group of experiments is to quantitatively verify whether the sentence type of the prompt affects the model’s understanding of the task and the quality of item semantic embeddings and to determine the optimal prompt sentence type for the MHLLM framework in e-commerce item embedding extraction scenarios.

Table A16. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different type prompts.

Prompts	Recall@5	Recall@10	Recall@50	Recall@200
Our Prompt	0.1218797	0.1668249	0.2232935	0.23137
Declarative Prompt	0.1492731	0.1890393	0.2214451	0.2258913
Interrogative prompt	0.1079416	0.1542023	0.2219613	0.227057

Table A17. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different type prompts.

Prompts	NDCG@5	NDCG@10	NDCG@50	NDCG@200
Our Prompt	0.0838967	0.0984844	0.111944	0.1131951
Declarative Prompt	0.1064191	0.1193999	0.1272388	0.1279217
Interrogative prompt	0.0736068	0.0736068	0.1046016	0.1053981

The experiment aims to quantitatively verify whether prompt sentence type affects the model’s task understanding and item semantic embedding quality and to determine the optimal prompt sentence type for the MHLLM in e-commerce item embedding extraction. Experimental results show that declarative prompts achieve the best performance in top-k recommendation metrics including Recall@5, Recall@10, NDCG@5, and NDCG@200. Imperative prompts rank second and exhibit more stable performance on high k-value metrics such as Recall@50 and Recall@200. Interrogative prompts perform the worst across most indicators with the lowest Recall and NDCG scores. This indicates that declarative prompts, which provide clear task context, are more conducive to guiding MHLLM to generate high-quality item embeddings and boosting top-k e-commerce recommendation accuracy, while imperative prompts are more competitive in long-tail recommendation scenarios.

In summary, the proposed task-specific prompt serves as an effective component that improves item representation quality and contributes to the overall accuracy of cross-modal recommendation in the MHLLM framework. We observe that more detailed prompts and declarative prompts yield further gains on top-k metrics, which aligns with the common understanding that LLMs are sensitive to instruction specificity and phrasing. The dedicated prompt mechanism we propose is demonstrably effective and can be further enhanced with more detailed instructions or declarative formulations, demonstrating that prompt design is critical to the MHLLM framework.

Appendix E.5. Ablation Experiments on [ITEM] Token

In the MHLLM, the [ITEM] token is not a fixed token ID defined in the vocabulary but is implemented as a trainable model parameter (initialized with a normal distribution

and supporting loading pre-trained vectors). It is injected by replacing the embedding vector of the last token of the item text during the item embedding generation phase; if the token is disabled, it degrades to mean pooling. Its core function is to aggregate the global semantics of the item text and generate fixed-dimensional item embeddings, serving as the core connection carrier between the Item-LLM and the User-LLM. Finally, it completes user–item matching training with NCE loss, supports precomputation of item embeddings and user–item retrieval during inference, and does not require modifying the LLM vocabulary/Tokenizer, making it compatible with various LLM architectures such as Llama, Mistral, BERT, etc.

Table A18. Recall performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different [ITEM] tokens.

[ITEM] Token	Recall@5	Recall@10	Recall@50	Recall@200
Trainable Token	0.1218797	0.1668249	0.2232935	0.23137
Mean Pooling Token	0.0914389	0.1257265	0.2085894	0.220446

Table A19. NDCG performance of MHLLM with Llama-3-74M as the LLM and BGE-VL-Large as the CLIP model on the rec_tmall_60K dataset, using different [ITEM] tokens.

[ITEM] Token	NDCG@5	NDCG@10	NDCG@50	NDCG@200
Trainable Token	0.0838967	0.0984844	0.111944	0.1131951
Mean Pooling Token	0.063306	0.0743317	0.0930013	0.094895

Experiments demonstrate that the trainable ITEM token significantly outperforms the mean pooling scheme across all Recall and NDCG metrics, which fully validates its core role in aggregating the global semantics of item text and enhancing user–item matching effectiveness. Meanwhile, this design requires no modifications to the LLM vocabulary or tokenizer and is compatible with various mainstream LLM architectures such as Llama.

Appendix F. Supplementary Study on the Adaptation of Multimodal Baselines (MMSBR and MMGCN)

To complement the main experiments and to explore the upper bounds of cross-modal recommendation under constrained data availability, we additionally evaluated two representative multimodal baselines: MMSBR [5] (Multimodal Session-Based Recommendation) and MMGCN [28] (Multimodal Graph Convolution Network). However, a direct, off-the-shelf application of these models to our datasets is infeasible, as their original architectures require specific input modalities and feature spaces (e.g., hierarchical category taxonomies, item pricing, and acoustic signals) that are absent from both the rec_tmall_60K and Pixel200K benchmarks.

Consequently, we performed necessary feature-space adaptations to enable a preliminary comparison. This appendix details the adaptation protocols and reports the corresponding results. We emphasize that the reported figures for MMSBR and MMGCN are solely intended for qualitative trend analysis. They must not be directly compared with the values reported in the original publications, nor do they serve as the primary evidence for the superiority of MHLLM. The main claims of this paper are firmly grounded in the fair and reproducible comparisons against HLLM, SASRec, and BERT4Rec, conducted under strictly identical experimental settings.

Appendix F.1. Feature-Space Adaptation Protocols

(a) Input Feature Reconstruction for MMSBR

The native MMSBR encoder relies on two explicit attribute embeddings—namely fine-grained category IDs and price values—to model contextual user intent. Since these fields are unavailable in our datasets, we implemented the following proxies:

- **Category Feature Proxy:** The raw category field in `rec_tmall_60K` provides only a single-level coarse ID. To approximate the semantic richness of a hierarchical category tree, we replaced the category embedding with a 768-dimensional [CLS] pooled vector extracted by a frozen Bert-base-Chinese encoder using the category name string.
- **Price Feature Proxy:** As the datasets contain no absolute price information, we introduced a Normalized Interaction Popularity score as a surrogate for price signals:

$$Price_{proxy}(i) = \frac{\log(1 + f_i)}{\max_{j \in I} \log(1 + f_j)}$$

where f_i denotes the interaction frequency of item i in the training set. This proxy assumes that frequently interacted items implicitly occupy a distinct preference distribution that, in absence of price, serves as a coarse behavioral calibrator.

It is critical to note that this substitution introduces a theoretical bias: popularity is generally positively correlated with engagement, whereas price typically exerts a negative influence on conversion propensity under fixed utility. Therefore, the adapted MMSBR is likely to learn a skewed preference representation that diverges from its original design.

(b) Modality Branch Pruning for MMGCN

The original MMGCN framework was designed for micro-video recommendation, integrating three complementary modalities: visual (frames), textual (titles/tags), and acoustic (audio waveforms). Since our e-commerce datasets are static and exclusively image-text based, we

- Removed the entire acoustic encoder branch along with its associated modality-interaction adjacency matrices.
- Retained only the visual branch (ResNet-50, initialized with ImageNet weights) and the textual branch (BERT-base), preserving the original user-item graph structure.
- We computed the inter-modal alignment loss solely between the visual and textual embeddings, omitting the acoustic-related fusion parameters.

As a consequence, the adapted MMGCN degrades to a dual-modal graph model, losing the complementary representational benefits of the acoustic channel. This pruning inevitably compromises the model's originally reported capacity and renders its absolute performance non-comparable with the full-fledged version.

Appendix F.2. Full Performance Reports on Adapted Baselines

The following four tables present the complete Recall and NDCG metrics for the adapted MMSBR and MMGCN across both datasets. For direct reference, the corresponding MHLLM results (excerpted from Tables 4–7 in the main text) are also included.

Table A20. Recall performance of adapted baselines vs. MHLLM on `rec_tmall_60K`.

Model	Recall@5	Recall@10	Recall@50	Recall@200
MMSBR	0.0078125	0.0078125	0.0156250	0.0312500
MMGCN	0.0012165	0.0012165	0.0042579	0.0095296
MHLLM1 (Ours)	0.1218797	0.1668249	0.2232935	0.23137

Table A21. NDCG performance of adapted baselines vs. MHLLM on rec_tmall_60K.

Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
MMSBR	0.0049291	0.0049291	0.0066801	0.0091790
MMGCN	0.0005771	0.0005771	0.0014573	0.0024468
MHLLM1 (Ours)	0.0859951	0.1009343	0.1133522	0.1146824

Table A22. Recall performance of adapted baselines vs. MHLLM on Pixel200K.

Model	Recall@5	Recall@10	Recall@50	Recall@200
MMSBR	0.0001302	0.0003255	0.0018229	0.0074219
MMGCN	0.0000324	0.0001295	0.0007446	0.0030696
MHLLM3 (Ours)	0.000230	0.000375	0.002075	0.006525

Table A23. NDCG performance of adapted baselines vs. MHLLM on Pixel200K.

Model	NDCG@5	NDCG@10	NDCG@50	NDCG@200
MMSBR	0.0000663	0.0001290	0.0004431	0.0012470
MMGCN	0.0000154	0.0000535	0.0002023	0.0006275
MHLLM3 (Ours)	0.0001358	0.0001817	0.0005412	0.0011952

Appendix F.3. Limitations, Interpretation Guidelines, and Concluding Remarks

The results presented in Appendix F are subject to several non-trivial confounders that must be carefully considered:

1. **Validity of Proxy Features:** The use of Normalized Interaction Popularity as a surrogate for price in MMSBR is theoretically fragile. Price and popularity often exhibit contradictory correlations with user preference (negative vs. positive). This proxy may introduce an inherent inductive bias that fundamentally alters the learned decision boundary, which explains the severely depressed absolute Recall values observed.
2. **Information Bottleneck from Modality Pruning:** The removal of the acoustic branch in MMGCN not only reduces model capacity but also eliminates the synergistic interactions among modalities that the original architecture was explicitly designed to capture. The adapted version thus constitutes a weakened baseline that underestimates the original MMGCN's potential.
3. **Long-tail Anomalies in Pixel200K:** In the extremely sparse Pixel200K setting, the adapted MMSBR marginally surpasses MHLLM on long-tail metrics (Recall@200 and NDCG@200). However, as established in large-scale industrial A/B testing [52–54], long-tail metrics exhibit poor correlation with actual online KPIs (e.g., CTR and GMV), as over 90% of user engagements occur within the top-10 ranked items. Therefore, the slight advantage on @200 does not indicate practical superiority.
4. **Core Evaluation Standard:** We re-emphasize that the top-k ranking performance, particularly Recall@5 and NDCG@10, remains the gold standard for evaluating recommendation discriminability. On these core metrics, MHLLM consistently and substantially outperforms the adapted baselines across all settings.

In summary, the findings of this appendix are presented solely to document our complete experimental exploration and to ensure reproducibility. They should be interpreted as qualitative indicators of model behavior under feature adaptation rather than as quantitative benchmarks. All substantive conclusions regarding state-of-the-art comparison in the main paper are exclusively drawn from the fair and unmodified evaluations against HLLM, SASRec, and BERT4Rec.

References

1. Statista. Global Retail E-Commerce Sales 2022–2030. 2026. Available online: <https://www.statista.com/statistics/379046/worldwide-retail-e-commerce-sales/> (accessed on 22 July 2026).
2. Wang, F.; Wang, M.; Zheng, Y.; Jin, J.; Pan, Y. Consumer Vigilance and Choice Overload in Online Shopping. *Int. J. Electron. Commer.* **2021**, *25*, 364–390. [CrossRef]
3. Zhang, Y.; Wang, R.; Cheng, D.Z.; Yao, T.; Yi, X.; Hong, L.; Caverlee, J.; Chi, E.H. Empowering Long-Tail Item Recommendation Through Cross Decoupling Network (CDN). In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Long Beach, CA, USA*; ACM: New York, NY, USA, 2023; pp. 5608–5617.
4. IBM. 2024 Consumer Study: Revolutionize Retail with AI Everywhere. Available online: <https://www.ibm.com/downloads/documents/us-en/10a99803fd2fdd87> (accessed on 22 July 2026).
5. Zhang, X.; Xu, B.; Ma, F.; Li, C.; Yang, L.; Lin, H. Beyond Co-Occurrence: Multi-Modal Session-Based Recommendation. *IEEE Trans. Knowl. Data Eng.* **2024**, *36*, 1450–1462. [CrossRef]
6. McKinsey & Company. The Value of Getting Personalization Right—Or Wrong—Is Multiplying. 12 November 2021. Available online: <https://www.mckinsey.com/capabilities/growth-marketing-and-sales/our-insights/the-value-of-getting-personalization-right-or-wrong-is-multiplying> (accessed on 22 July 2026).
7. Pazzani, M.J.; Billsus, D. Content-Based Recommendation Systems. In *The Adaptive Web*; Brusilovsky, P., Kobsa, A., Nejdl, W., Eds.; Springer: Berlin, Heidelberg, 2007; Volume 4321, pp. 325–341.
8. Sarwar, B.M.; Karypis, G.; Konstan, J.A.; Riedl, J. Item-Based Collaborative Filtering Recommendation Algorithms. In *Proceedings of the 10th International Conference on World Wide Web, Hong Kong, China, 1–5 May 2001*; ACM: New York, NY, USA, 2001; pp. 285–295.
9. Schafer, J.B.; Frankowski, D.; Herlocker, J.L.; Sen, S. Collaborative Filtering Recommender Systems. In *The Adaptive Web*; Brusilovsky, P., Kobsa, A., Nejdl, W., Eds.; Springer: Berlin, Heidelberg, 2007; Volume 4321, pp. 291–324.
10. Krizhevsky, A.; Sutskever, I.; Hinton, G.E. Imagenet classification with deep convolutional neural networks. *Adv. Neural Inf. Process. Syst.* **2012**, *25*, 1097–1105.
11. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; Computer Vision Foundation: Las Vegas, NV, USA, 2016; pp. 770–778.
12. Kang, W.-C.; McAuley, J. Self-Attentive Sequential Recommendation. In *Proceeding of the 2018 IEEE International Conference on Data Mining (ICDM)*; IEEE: Singapore, 2018; pp. 197–206.
13. Chen, J.; Chi, L.; Peng, B.; Yuan, Z. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. *arXiv* **2024**, arXiv:2409.12740.
14. Cheng, Y.; Pan, Y.; Zhang, J.; Ni, Y.; Sun, A.; Yuan, F. An Image Dataset for Benchmarking Recommender Systems with Raw Pixels. *arXiv* **2023**, arXiv:2309.06789.
15. Radford, A.; Kim, J.W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; et al. Learning Transferable Visual Models from Natural Language Supervision. *arXiv* **2021**, arXiv:2103.00020.
16. Lyu, H.; Jiang, S.; Zeng, H.; Xia, Y.; Luo, J. LLM-Rec: Personalized Recommendation via Prompting Large Language Models. *arXiv* **2023**, arXiv:2307.15780.
17. Sun, F.; Liu, J.; Wu, J.; Pei, C.; Lin, X.; Ou, W.; Jiang, P. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, Beijing, China, 3–7 November 2019*; Association for Computing Machinery: New York, NY, USA, 2019; pp. 1441–1450.
18. Tianchi Alibaba Cloud. Rec-Tmall Dataset: Brand and Item Data from Tmall Recommendation Contest. 2022. Available online: <https://tianchi.aliyun.com/dataset/140281> (accessed on 22 July 2026).
19. Zhang, S.; Yao, L.; Sun, A.; Tay, Y. Deep Learning Based Recommender System: A Survey and New Perspectives. *ACM Comput. Surv.* **2019**, *52*, 1–38. [CrossRef]
20. Covington, P.; Adams, J.; Sargin, E. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems, Boston, MA, USA, 15–19 September 2016*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 191–198.
21. Cheng, H.-T.; Koc, L.; Harmsen, J.; Shaked, T.; Chandra, T.; Aradhye, H.; Anderson, G.; Corrado, G.; Chai, W.; Ispir, M.; et al. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems, Boston, MA, USA, 15 September 2016*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 7–10.
22. Zhou, G.; Song, C.; Zhu, X.; Fan, S.; Zhu, Y.; Ma, H.; Yan, Y.; Jin, J.; Li, H.; Gai, K. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, London, UK, 19–23 August 2018*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 1059–1068.
23. Zhou, G.; Zhu, X.; Song, C.; Fan, S.; Zhu, Y.; Ma, H.; Yan, Y.; Jin, J.; Li, H.; Gai, K. Deep Interest Evolution Network for Click-Through Rate Prediction. *Proc. AAAI Conf. Artif. Intell.* **2019**, *33*, 5941–5948. [CrossRef]

24. Hidasi, B.; Karatzoglou, A.; Baltrunas, L.; Tikk, D. Session-Based Recommendations with Recurrent Neural Networks. *arXiv* **2016**, arXiv:1511.06939.
25. Scarselli, F.; Gori, M.; Tsoi, A.C.; Hagenbuchner, M.; Monfardini, G. The Graph Neural Network Model. *IEEE Trans. Neural Netw.* **2009**, *20*, 61–80. [[CrossRef](#)] [[PubMed](#)]
26. Ying, R.; He, R.; Chen, K.; Eksombatchai, P.; Hamilton, W.L.; Leskovec, J. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, London, UK, 19–23 August 2018*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 974–983.
27. Guo, Q.; Zhuang, X.; Zhang, Y.; Sun, Y.; Liu, Z.; Xie, X. A Survey on Knowledge Graph-Based Recommender Systems. *IEEE Trans. Knowl. Data Eng.* **2022**, *34*, 3549–3568. [[CrossRef](#)]
28. Wei, Y.; Wang, X.; Nie, L.; He, X.; Hong, R.; Chua, T.-S. MMGCN: Multi-Modal Graph Convolution Network for Personalized Recommendation of Micro-Video. In *Proceedings of the 27th ACM International Conference on Multimedia (MM '19), Nice, France, 21–25 October 2019*; Association for Computing Machinery: New York, NY, USA, 2019; pp. 1437–1445.
29. Guo, L.; Zhou, S.; Tang, H.; Zheng, X.; Luo, Y. Multi-Behavior Hypergraph Contrastive Learning for Session-Based Recommendation. *IEEE Trans. Knowl. Data Eng.* **2025**, *37*, 1325–1338. [[CrossRef](#)]
30. Wu, L.; Zheng, Z.; Qiu, Z.; Wang, H.; Gu, H.; Shen, T.; Qin, C.; Zhu, C.; Zhu, H.; Liu, Q.; et al. A Survey on Large Language Models for Recommendation. *arXiv* **2023**, arXiv:2305.19860.
31. Zhou, J.; Xiong, Y.; Liu, Z.; Liu, Z.; Xiao, S.; Wang, Y.; Zhao, B.; Zhang, C.J.; Lian, D. MegaPairs: Massive Data Synthesis for Universal Multimodal Retrieval. *arXiv* **2024**, arXiv:2412.14475.
32. Zhang, K.; Luan, Y.; Hu, H.X.; Lee, K.; Qiao, S.; Chen, W.H.; Su, Y.; Chang, M.-W. MagicLens: Self-Supervised Image Retrieval with Open-Ended Instructions. *arXiv* **2024**, arXiv:2403.19651.
33. Gadre, S.Y.; Ilharco, G.; Fang, A.; Hayase, J.; Smyrnis, G.; Nguyen, T.; Marten, R.; Wortsman, M.; Ghosh, D.; Zhang, J.; et al. DataComp: In Search of the Next Generation of Multimodal Datasets. *arXiv* **2023**, arXiv:2304.14108.
34. Sun, Q.; Fang, Y.; Wu, L.; Wang, X.; Cao, Y. EVA-CLIP: Improved Training Techniques for CLIP at Scale. *arXiv* **2023**, arXiv:2303.15389.
35. Oquab, M.; Darcet, T.; Moutakanni, T.; Vo, H.; Szafraniec, M.; Khalidov, V.; Fernandez, P.; Haziza, D.; Massa, F.; El-Nouby, A.; et al. DINOv2: Learning Robust Visual Features without Supervision. *arXiv* **2023**, arXiv:2304.07193.
36. Chen, T.; Kornblith, S.; Norouzi, M.; Hinton, G. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning*; PMLR: Cambridge, MA, USA, 2020; Volume 119, pp. 1597–1607.
37. Chen, Z.; Wu, J.; Wang, W.; Su, W.; Chen, G.; Xing, S.; Zhong, M.; Zhang, Q.; Zhu, X.; Lu, L.; et al. InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: New York, NY, USA, 2024; pp. 24185–24198.
38. Meta Llama Team. The Llama 3 Herd of Models. *arXiv* **2024**, arXiv:2407.21783.
39. Jiang, Z.; Meng, R.; Yang, X.; Yavuz, S.; Zhou, Y.; Chen, W. VLM2Vec: Training Vision-Language Models for Massive Multimodal Embedding Tasks. *arXiv* **2024**, arXiv:2410.05160.
40. van den Oord, A.; Li, Y.; Vinyals, O. Representation Learning with Contrastive Predictive Coding. *arXiv* **2018**, arXiv:1807.03748.
41. Wang, Q.; Li, J.; Wang, S.; Xing, Q.; Niu, R.; Kong, H.; Li, R.; Long, G.; Chang, Y.; Zhang, C. Towards Next-Generation LLM-Based Recommender Systems: A Survey and Beyond. *arXiv* **2024**, arXiv:2410.19744.
42. Kumar, R.; Dey, T.; Das, S. Empowering Recommendations: Survey of LLM-Based Recommendation Systems. In *Proceedings of the 8th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech), Kolkata, India, 31 January–2 February 2025*; IEEE: New York, NY, USA, 2025; pp. 525–530.
43. Zhang, P.; Zeng, G.; Wang, T.; Lu, W. TinyLlama: An Open-Source Small Language Model. *arXiv* **2024**, arXiv:2401.02385.
44. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*; Association for Computational Linguistics: Minneapolis, MN, USA, 2019; pp. 4171–4186.
45. Rumelhart, D.E.; Hinton, G.E.; Williams, R.J. Learning Representations by Back-Propagating Errors. *Nature* **1986**, *323*, 533–536. [[CrossRef](#)]
46. Hendrycks, D.; Gimpel, K. Gaussian Error Linear Units (GELUs). *arXiv* **2016**, arXiv:1606.08415.
47. Qwen Team. Qwen2.5 Technical Report. *arXiv* **2025**, arXiv:2412.15115.
48. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv* **2023**, arXiv:2302.13971.
49. Zaharia, M.; Xin, R.S.; Wendell, P.; Das, T.; Armbrust, M.; Dave, A.; Meng, X.; Rosen, J.; Venkataraman, S.; Franklin, M.J.; et al. Apache Spark: A Unified Engine for Big Data Processing. *Commun. ACM* **2016**, *59*, 56–65. [[CrossRef](#)]
50. Rasley, J.; Rajbhandari, S.; Ruwase, O.; He, Y. DeepSpeed: System Optimizations Enable Training Deep Learning Models with over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*; Association for Computing Machinery: New York, NY, USA, 2020; pp. 3551–3561. [[CrossRef](#)]

51. Li, S.; Zhao, Y.; Varma, R.; Salpekar, O.; Noordhuis, P.; Li, T.; Paszke, A.; Smith, J.; Vaughan, B.; Damania, P.; et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proc. VLDB Endow.* **2020**, *13*, 3005–3018. [[CrossRef](#)]
52. Ma, Y.; Wang, Z.; Yan, Y.; Wu, J.; Jiang, Y.; Li, L.; Chen, W.; Huang, J. Simple but Efficient: A Multi-Scenario Nearline Retrieval Framework for Recommendation on Taobao. *arXiv* **2024**, arXiv:2408.00247.
53. Jadon, A.; Patil, A. A Comprehensive Survey of Evaluation Techniques for Recommendation Systems. *arXiv* **2023**, arXiv:2312.16015.
54. Valcarce, D.; Bellogín, A.; Parapar, J.; Castells, P. On the Robustness and Discriminative Power of Information Retrieval Metrics for Top-N Recommendation. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys 2018), Vancouver, BC, Canada, 2–7 October 2018*; Association for Computing Machinery: New York, NY, USA, 2018; pp. 260–268.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.